

Universidad ORT Uruguay
Facultad de Ingeniería
Escuela de Tecnología

OBLIGATORIO PROGRAMACIÓN 2



Franco Beloqui – 360021



Emiliano Baston- 163914

N2E

Docente: ANDRÉS URETA

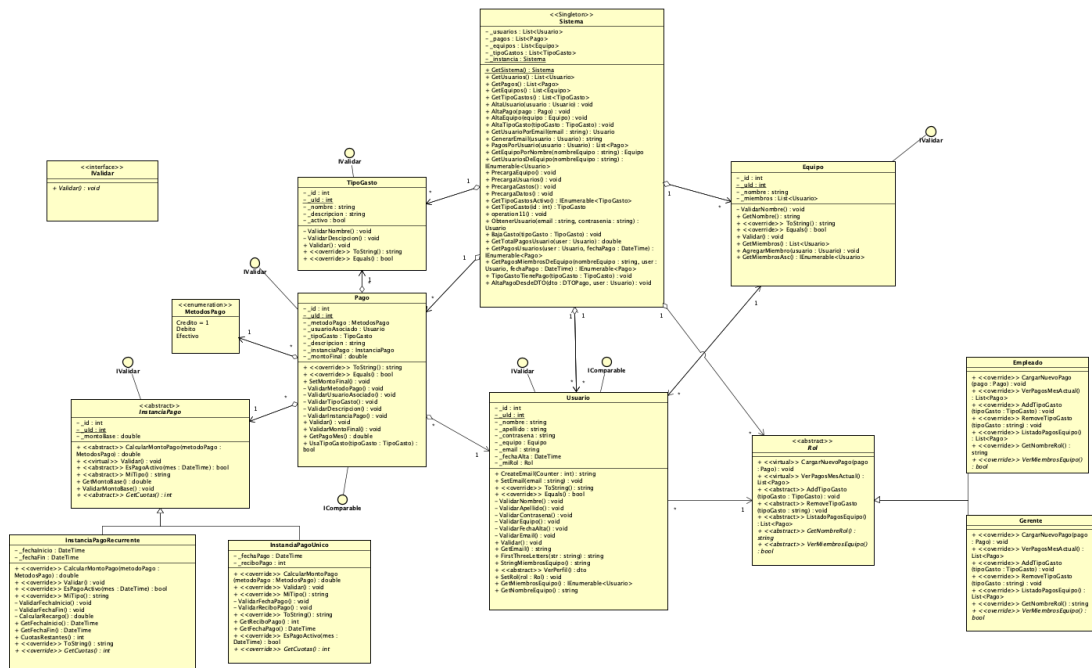
Analista Programador

Fecha de entrega del documento: 27-11-2025

1. Diagrama de clases del Dominio	4
2. Tabla de datos precargados	5
2.1 Precarga Equipos	5
2.2 Precarga Usuarios	5
2.3 Precarga Gastos	6
2.4 Precarga Pagos	6
2.4.1 Recurrentes sin Fecha de Fin	6
2.4.2 Recurrentes con Fecha de Fin en el futuro	7
2.4.3 Recurrentes con Fecha de Fin en el pasado	8
2.4.4 Pagos único.	9
3. Código fuente	10
3.1 Dominio	10
3.1.1 Clase Sistema	10
3.1.2 Usuarios	27
3.1.2.1 Clase Usuario	27
3.1.2.2 Clase Equipo	32
3.1.2.3 Clase DTOUser	34
3.1.3 Pagos	35
3.1.3.1 Clase DTOPagos	35
3.1.3.2 Clase Instancia Pago (Abstracta)	38
3.1.3.3 Clase Instancia de pago Recurrente	39
3.1.3.4 Clase Instancia de pago Unico	41
3.1.3.5 Metodos de pago Enum	43
3.1.3.6 Clase Pago	43
3.1.3.7 Clase Tipo de Gasto	47
3.1.4 Roles	48
3.1.4.1 Clase Rol	48
3.1.4.2 Clase Rol Gerente	49
3.1.4.3 Clase Rol Empleado	50
3.1.1.5 IValidar (Interface)	51
3.2 Codigo MVC	52
3.2.1 Controllers	52
3.2.1.1 Empleado Controller	52
3.2.1.2 Gerente Controller	55
3.2.1.3 Home Controller	61
3.2.1.4 Usuario Controller	61
3.3.2 Filtros	63
3.3.2.1 Log Action Filter	63
3.3.2.2 Rol Action Filter	64
3.3.3 Vistas	65
3.3.3.1 Empleado	65
3.3.3.1.1 Alta de Pago	65
3.3.3.1.2 Index	69
3.3.3.1.3 Mis Pagos	70

3.2.3.1.4 Perfil	74
3.3.3.2 Gerente	75
3.3.3.2.1 Alta de Gasto	75
3.3.3.2.2 Alta de Pago	77
3.3.3.2.3 Eliminar Gasto	80
3.3.3.2.4 Gastos	81
3.3.3.2.5 Index	83
3.3.3.2.6 Pagos	86
3.3.3.2.7 Perfil	92
3.3.3.3 Home	94
3.3.3.3.1 Index	94
3.3.3.4 Shared	94
3.3.3.4.1 Layout	94
4. Prompts AI	99
5. Diagrama de casos de uso	99
6. Plantilla de casos de Prueba	100
6.1 Login	100
6.2 Crear Nuevo Pago	100
6.3 Listado de Pagos	102
6.4 Nuevo tipo de Gasto	102
6.5 Eliminar Gasto	103
7. Link Azure	103

pkg



2. Tabla de datos precargados

2.1 Precarga Equipos

ID	Nombre del Equipo
1	Contabilidad
2	Finanzas
3	Tesorería
4	Auditoría
5	Ventas

2.2 Precarga Usuarios

ID	Nombre	Apellido	Contraseña	Equipo	Rol
1	Juana	Melcer	12345678	Contabilidad	Gerente
2	Ana	Perez	12345678	Contabilidad	Empleado
3	Lucas	Garcia	12345678	Contabilidad	Gerente
4	Martina	Rodriguez	12345678	Contabilidad	Empleado
5	Santiago	Fernandez	12345678	Contabilidad	Gerente
6	Valentina	Gomez	12345678	Contabilidad	Empleado
7	Mateo	Suarez	12345678	Finanzas	Gerente
8	Camila	Lopez	12345678	Finanzas	Empleado
9	Joaquin	Alvarez	12345678	Finanzas	Gerente
10	Sofia	Martinez	12345678	Finanzas	Empleado
11	Tomas	Pereira	12345678	Finanzas	Gerente
12	Isabella	Santos	12345678	Tesorería	Empleado
13	Agustin	Cabrera	12345678	Tesoreria	Gerente
14	Mia	Torres	12345678	Tesoreria	Empleado
15	Facundo	Silva	12345678	Tesoreria	Empleado
16	Julia	Ramos	12345678	Tesoreria	Gerente
17	Nicolas	Castro	12345678	Auditoria	Empleado
18	Florencia	Vega	12345678	Auditoria	Gerente
19	Andres	Morales	12345678	Auditoria	Empleado
20	Carolina	Ruiz	12345678	Auditoria	Gerente
21	Bruno	Herrera	12345678	Auditoria	Empleado
22	Lucia	Acosta	12345678	Auditoria	Gerente

2.3 Precarga Gastos

ID	Tipo de Gasto	Descripción
1	Alquiler	Pago mensual por el alquiler de la oficina
2	Servicios	Pago de servicios como luz, agua, internet, etc.
3	Sueldos	Pago de sueldos a los empleados
4	Materiales de oficina	Compra de materiales de oficina como papel, bolígrafos, etc.
5	Publicidad	Gastos en publicidad y marketing
6	Viajes	Gastos en viajes de negocios

2.4 Precarga Pagos

2.4.1 Recurrentes sin Fecha de Fin

Metodo de pago	Usuario	Tipo de gasto	Descripción	Instancia de pago	Fecha de inicio	Fecha de Fin
Credito	Juana Melcer	Alquiler	Alquiler mensual	Recurrente	2024-11-26	Indefinido
Debito	Ana Perez	Servicios	Servicios (UTE/ANTEL)	Recurrente	2025-02-26	Indefinido
Credito	Lucas Garcia	Sueldos	Sueldos	Recurrente	2025-01-26	Indefinido
Credito	Martina Rodriguez	Materiales de oficina	Mantenimiento	Recurrente	2025-05-26	Indefinido
Debito	Santiago Fernandez	Publicidad	Internet	Recurrente	2025-03-26	Indefinido
Credito	Valentina Gomez	Viajes	Limpieza	Recurrente	2025-06-26	Indefinido

2.4.2 Recurrentes con Fecha de Fin en el futuro

Metodo de pago	Usuario	Tipo de gasto	Descripción	Instancia de pago	Fecha de inicio	Fecha de Fin
Credito	Mateo Suarez	Alquiler	Licencia software	Recurrente	2025-07-26	2026-07-26
Debito	Camila Lopez	Servicios	Seguro flota	Recurrente	2025-04-26	2026-04-26
Credito	Joaquin Alvarez	Sueldos	Alquiler depósito	Recurrente	2025-09-26	2026-09-26
Credito	Sofia Martinez	Materiales de oficina	Servicio guardia	Recurrente	2025-08-26	2026-03-26
Debito	Tomas Pereira	Publicidad	Telefonía móvil	Recurrente	2025-10-26	2026-11-26
Credito	Isabella Santos	Viajes	Mantenimiento HVAC	Recurrente	2025-05-26	2026-01-26
Credito	Agustin Cabrera	Alquiler	Soporte técnico	Recurrente	2025-06-26	2026-05-26
Debito	Mia Torres	Servicios	Licencias antivirus	Recurrente	2025-03-26	2026-02-26
Credito	Facundo Silva	Sueldos	Estacionamiento	Recurrente	2025-09-26	2026-06-26

2.4.3 Recurrentes con Fecha de Fin en el pasado

Metodo de pago	Usuario	Tipo de gasto	Descripción	Instancia de pago	Fecha de inicio	Fecha de fin
Credito	Juana Melcer	Materiales de oficina	Campaña marketing	Recurrente	2024-09-26	2025-05-26
Debito	Ana Perez	Publicidad	Contrato limpieza	Recurrente	2024-11-26	2025-08-26
Credito	Lucas Garcia	Viajes	Consultoría RRHH	Recurrente	2025-02-26	2025-10-26
Credito	Martina Rodriguez	Alquiler	Capacitación personal	Recurrente	2025-01-26	2025-09-26
Debito	Santiago Fernandez	Servicios	Servicio backup	Recurrente	2024-12-26	2025-06-26
Credito	Valentina Gomez	Sueldos	Alquiler sala reuniones	Recurrente	2025-04-26	2025-07-26
Credito	Mateo Suarez	Materiales de oficina	Publicidad trimestral	Recurrente	2025-03-26	2025-09-26
Debito	Camila Lopez	Publicidad	Soporte ERP	Recurrente	2024-08-26	2025-02-26
Debito	Joaquin Alvarez	Viajes	Servicio jardinería	Recurrente	2024-10-26	2025-03-26
Credito	Sofia Martinez	Alquiler	Monitoreo cámaras	Recurrente	2025-05-26	2025-10-26

2.4.4 Pagos único.

Metodo de pago	Usuario	Tipo de gasto	Descripción	Instancia de pago	Fecha de inicio	Recibo
Efectivo	Tomas Pereira	Servicios	Compra insumos oficina	Unico	2025-11-01	2001
Debito	Isabella Santos	Sueldos	Flete puntual mercadería	Unico	2025-11-16	2002
Efectivo	Agustin Cabrera	Materiales de oficina	Reparación impresora	Unico	2025-10-17	2003
Efectivo	Mia Torres	Publicidad	Catering reunión clientes	Unico	2025-11-21	2004
Debito	Facundo Silva	Viajes	Taxi traslado urgente	Unico	2025-11-23	2005
Efectivo	Juana Melcer	Alquiler	Reposición sillas	Unico	2025-09-27	2006
Efectivo	Ana Perez	Servicios	Papelería y timbres	Unico	2025-11-11	2007
Debito	Lucas Garcia	Sueldos	Carga combustible única	Unico	2025-11-19	2008
Debito	Martina Rodriguez	Materiales de oficina	Compra luces LED	Unico	2025-11-06	2009
Efectivo	Santiago Fernandez	Publicidad	Pago peaje excepcional	Unico	2025-11-25	2010
Debito	Valentina Gomez	Viajes	Servicio plomería	Unico	2025-11-13	2011
Debito	Mateo Suarez	Alquiler	Compra monitor 27	Unico	2025-10-24	2012
Efectivo	Camila Lopez	Servicios	Cafetera nueva	Unico	2025-11-08	2013
Debito	Joaquin Alvarez	Sueldos	Reemplazo neumático	Unico	2025-11-04	2014
Efectivo	Sofia Martinez	Materiales de oficina	Servicio desinfección	Unico	2025-10-30	2015
Efectivo	Tomas Pereira	Publicidad	Compra matafuegos	Unico	2025-10-12	2016
Debito	Ana Perez	Viajes	Reparación notebook	Unico	2025-11-14	2017

3. Código fuente

3.1 Dominio

3.1.1 Clase Sistema

```
using Clases.Pagos;
using Clases.Roles;
using Clases.Usuarios;

namespace ObligatorioP2
{
    public class Sistema
    {
        private List<Usuario> Usuarios;
        private List<Pago> Pagos;
        private List<Equipo> Equipos;
        private List<TipoGasto> TipoGastos;
        private static Sistema Instancia;

        private Sistema()
        {
            Usuarios = new List<Usuario>();
            Pagos = new List<Pago>();
            Equipos = new List<Equipo>();
            TipoGastos = new List<TipoGasto>();
            PrecargaDatos();
        }

        public static Sistema GetSistema()
        {
            if (Instancia == null)
            {
                Instancia = new Sistema();
            }
            return Instancia;
        }

        public List<Usuario> GetUsuarios()
        {
            return Usuarios;
        }
    }
}
```

```

public List<Pago> GetPagos()
{
    return Pagos;
}
public List<Equipo> GetEquipos()
{
    return Equipos;
}
public IEnumerable<TipoGasto> GetTipoGastos()
{
    return TipoGastos;
}
public IEnumerable<TipoGasto> GetTipoGastosActivos()
{
    List<TipoGasto> tipos = new List<TipoGasto>();
    foreach(TipoGasto t in TipoGastos)
    {
        if (t.Activo)
        {
            tipos.Add(t);
        }
    }
    return tipos;
}
public void AltaUsuario(Usuario usuario)
{
    try
    {
        if (usuario == null)
        {
            throw new Exception("El usuario no puede ser
nulo.");
        }
        if (Usuarios.Contains(usuario))
        {
            throw new Exception("El usuario ya existe.");
        }
        usuario.Validar();
        usuario.SetEmail(GenerarEmail(usuario));
    }
    catch (Exception ex)
    {
        throw new Exception("Error al validar el usuario: "
+ ex.Message);
    }
    Usuarios.Add(usuario);
}
public void AltaEquipo(Equipo equipo)

```

```

    {
        try
        {
            if (equipo == null)
            {
                throw new Exception("El equipo no puede ser
nulo.");
            }
            if (Equipos.Contains(equipo))
            {
                throw new Exception("El equipo ya existe.");
            }
            equipo.Validar();
        }
        catch (Exception ex)
        {
            throw new Exception("Error al validar el equipo: "
+ ex.Message);
        }
        Equipos.Add(equipo);
    }
    public void AltaTipoGasto(TipoGasto tipoGasto)
    {
        try
        {
            if (tipoGasto == null)
            {
                throw new Exception("El tipo de gasto no puede
ser nulo.");
            }
            tipoGasto.Validar();

            if (TipoGastos.Contains(tipoGasto))
            {
                TipoGasto t =
GetTipoGastoByName(tipoGasto.Nombre);
                if (t.Activo) {
                    throw new Exception("El tipo de gasto ya
existe.");
                }
                else {
                    t.Activo = true;
                    return;
                }
            }

        }
        catch (Exception ex)
    }

```

```

        {
            throw new Exception("Error: " + ex.Message);
        }
        TipoGastos.Add(tipoGasto);
    }
    public void AltaPago(Pago pago)
    {
        try
        {
            if (pago == null)
            {
                throw new Exception("El pago no puede ser nulo.");
            }
            if (Pagos.Contains(pago))
            {
                throw new Exception("El pago ya existe.");
            }
            pago.Validar();
        }
        catch (Exception ex)
        {
            throw new Exception("Error al validar el pago: " + ex.Message);
        }
        Pagos.Add(pago);
    }
    public Usuario GetUsuarioPorEmail(string email)
    {
        if (string.IsNullOrEmpty(email))
        {
            throw new Exception("El email no puede estar vacío.");
        }
        if (!email.Contains("@") || !email.Contains("."))
        {
            throw new Exception("El email ingresado no es válido.");
        }
        if (email.Length < 5)
        {
            throw new Exception("El email debe tener al menos 5 caracteres.");
        }
        if (!email.Contains("laEmpresa"))
        {

```

```

        throw new Exception("El email debe pertenecer a la
empresa (debe contener 'laEmpresa').");
    }
    foreach (Usuario usuario in Usuarios)
    {
        if (usuario.Email == email)
        {
            return usuario;
        }
    }
    return null;
}

public string GenerarEmail(Usuario usuario)
{
    int counter = 0;
    string email = null;
    bool flag = false;

    while (!flag)
    {
        email = usuario.CreateEmail(counter);

        if (GetUsuarioPorEmail(email) == null)
        {
            flag = true;
        }
        counter++;
    }
    return email;
}

public List<Pago> PagosPorUsuario(Usuario user)
{
    List<Pago> lisAux = new List<Pago>();
    foreach (Pago p in Pagos)
    {
        if (p.UsuarioAsociado.Email == user.Email)
        {
            lisAux.Add(p);
        }
    }
    return lisAux;
}

public TipoGasto GetTipoGasto(int id)
{
    foreach (TipoGasto tipo in TipoGastos)
    {

```

```

        if (tipo.Id == id)
        {
            return tipo;
        }
    }
    return null;
}

public TipoGasto GetTipoGastoByName(string name)
{
    foreach (TipoGasto tipo in TipoGastos)
    {
        if (tipo.Nombre == name)
        {
            return tipo;
        }
    }
    return null;
}

public Equipo GetEquipoPorNombre(string nombre)
{
    if (string.IsNullOrEmpty(nombre))
    {
        throw new Exception("El nombre del equipo no puede
estar vacío.");
    }
    if (nombre.Length < 3)
    {
        throw new Exception("El nombre del equipo debe
tener al menos 3 caracteres.");
    }

    foreach (Equipo equipo in Equipos)
    {
        if (equipo.Nombre.ToLower() == nombre.ToLower())
        {
            return equipo;
        }
    }
    return null;
}

public IEnumerable<Usuario> GetUsuariosDeEquipo(string
nombreEquipo)
{
    Equipo equipo = GetEquipoPorNombre(nombreEquipo);
    if (equipo == null)
    {

```

```

        throw new Exception("El equipo no existe.");
    }

    return equipo.GetMiembros();
}

public void PrecargaDatos()
{
    PrecargaEquipo();
    PrecargaUsuarios();
    PrecargaGastos();
    PrecargaPagos();
}

public void PrecargaEquipo()
{
    Equipo contabilidad = new Equipo("Contabilidad");
    AltaEquipo(contabilidad);

    Equipo finanzas = new Equipo("Finanzas");
    AltaEquipo(finanzas);

    Equipo tesoreria = new Equipo("Tesoreria");
    AltaEquipo(tesoreria);

    Equipo auditoria = new Equipo("Auditoria");
    AltaEquipo(auditoria);

    Equipo ventas = new Equipo("Ventas");
    AltaEquipo(ventas);
}

public void PrecargaUsuarios()
{
    // Equipos existentes
    Equipo contabilidad = GetEquipoPorNombre("Contabilidad");
    Equipo finanzas = GetEquipoPorNombre("Finanzas");
    Equipo tesoreria = GetEquipoPorNombre("Tesoreria");
    Equipo auditoria = GetEquipoPorNombre("Auditoria");

    Rol Gerente = new RolGerente();
    Rol Empleado = new RolEmpleado();
    // ---- Contabilidad (6) ----
    Usuario usuario1 = new Usuario("Juana", "Melcer",
"12345678", contabilidad, Gerente);
    AltaUsuario(usuario1);
    contabilidad.AgregarMiembro(usuario1);
}

```



```
    Usuario usuario2 = new Usuario("Ana", "Perez", "12345678",
contabilidad, Empleado);
    AltaUsuario(usuario2);
    contabilidad.AgregarMiembro(usuario2);

    Usuario usuario3 = new Usuario("Lucas", "Garcia", "12345678",
contabilidad, Gerente);
    AltaUsuario(usuario3);
    contabilidad.AgregarMiembro(usuario3);

    Usuario usuario4 = new Usuario("Martina", "Rodriguez",
"12345678", contabilidad, Empleado);
    AltaUsuario(usuario4);
    contabilidad.AgregarMiembro(usuario4);

    Usuario usuario5 = new Usuario("Santiago", "Fernandez",
"12345678", contabilidad, Gerente);
    AltaUsuario(usuario5);
    contabilidad.AgregarMiembro(usuario5);

    Usuario usuario6 = new Usuario("Valentina", "Gomez",
"12345678", contabilidad, Empleado);
    AltaUsuario(usuario6);
    contabilidad.AgregarMiembro(usuario6);

    // ---- Finanzas (5) ----
    Usuario usuario7 = new Usuario("Mateo", "Suarez", "12345678",
finanzas, Gerente);
    AltaUsuario(usuario7);
    finanzas.AgregarMiembro(usuario7);

    Usuario usuario8 = new Usuario("Camila", "Lopez", "12345678",
finanzas, Empleado);
    AltaUsuario(usuario8);
    finanzas.AgregarMiembro(usuario8);

    Usuario usuario9 = new Usuario("Joaquin", "Alvarez",
"12345678", finanzas, Gerente);
    AltaUsuario(usuario9);
    finanzas.AgregarMiembro(usuario9);

    Usuario usuario10 = new Usuario("Sofia", "Martinez",
"12345678", finanzas, Empleado);
    AltaUsuario(usuario10);
    finanzas.AgregarMiembro(usuario10);
```

```
    Usuario usuario11 = new Usuario("Tomas", "Pereira", "12345678",
finanzas, Gerente);
    AltaUsuario(usuario11);
    finanzas.AgregarMiembro(usuario11);

    // ---- Tesoreria (5) ----
    Usuario usuario12 = new Usuario("Isabella", "Santos",
"12345678", tesoreria, Empleado);
    AltaUsuario(usuario12);
    tesoreria.AgregarMiembro(usuario12);

    Usuario usuario13 = new Usuario("Agustin", "Cabrera",
"12345678", tesoreria, Gerente);
    AltaUsuario(usuario13);
    tesoreria.AgregarMiembro(usuario13);

    Usuario usuario14 = new Usuario("Mia", "Torres", "12345678",
tesoreria, Empleado);
    AltaUsuario(usuario14);
    tesoreria.AgregarMiembro(usuario14);

    Usuario usuario15 = new Usuario("Facundo", "Silva", "12345678",
tesoreria, Empleado);
    AltaUsuario(usuario15);
    tesoreria.AgregarMiembro(usuario15);

    Usuario usuario16 = new Usuario("Julia", "Ramos", "12345678",
tesoreria, Gerente);
    AltaUsuario(usuario16);
    tesoreria.AgregarMiembro(usuario16);

    // ---- Auditoria (5) ----
    Usuario usuario17 = new Usuario("Nicolas", "Castro",
"12345678", auditoria, Empleado);
    AltaUsuario(usuario17);
    auditoria.AgregarMiembro(usuario17);

    Usuario usuario18 = new Usuario("Florencia", "Vega",
"12345678", auditoria, Gerente);
    AltaUsuario(usuario18);
    auditoria.AgregarMiembro(usuario18);

    Usuario usuario19 = new Usuario("Andres", "Morales",
"12345678", auditoria, Empleado);
    AltaUsuario(usuario19);
    auditoria.AgregarMiembro(usuario19);
```

```

        Usuario usuario20 = new Usuario("Carolina", "Ruiz", "12345678",
auditoria, Gerente);
        AltaUsuario(usuario20);
        auditoria.AgregarMiembro(usuario20);

        Usuario usuario21 = new Usuario("Bruno", "Herrera", "12345678",
auditoria, Empleado);
        AltaUsuario(usuario21);
        auditoria.AgregarMiembro(usuario21);
        Usuario usuario22 = new Usuario("Lucia", "Acosta", "12345678",
auditoria, Gerente);
        AltaUsuario(usuario22);
        auditoria.AgregarMiembro(usuario22);

    }

    public void PrecargaGastos()
    {
        TipoGasto tipo1 = new TipoGasto("Alquiler", "Pago
mensual por el alquiler de la oficina");
        AltaTipoGasto(tipo1);
        TipoGasto tipo2 = new TipoGasto("Servicios", "Pago de
servicios como luz, agua, internet, etc.");
        AltaTipoGasto(tipo2);
        TipoGasto tipo3 = new TipoGasto("Sueldos", "Pago de
sueldos a los empleados");
        AltaTipoGasto(tipo3);
        TipoGasto tipo4 = new TipoGasto("Materiales de
oficina", "Compra de materiales de oficina como papel, bolígrafos,
etc.");
        AltaTipoGasto(tipo4);
        TipoGasto tipo5 = new TipoGasto("Publicidad", "Gastos
en publicidad y marketing");
        AltaTipoGasto(tipo5);
        TipoGasto tipo6 = new TipoGasto("Viajes", "Gastos en
viajes de negocios");
        AltaTipoGasto(tipo6);

    }

    public void PrecargaPagos()
    {
        // ===== 6 pagos RECURRENTES SIN fecha de fin (EndDate = null)
        =====

```

```

    AltaPago(new Pago(MetodosPago.Credito, Usuarios[0],
TipoGastos[0], "Alquiler mensual",new
InstanciaPagoRecurrente(28000, DateTime.Now.AddMonths(-12),
null)));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[1],
TipoGastos[1], "Servicios (UTE/ANTEL)",new
InstanciaPagoRecurrente(6500, DateTime.Now.AddMonths(-9),
null)));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[2],
TipoGastos[2], "Suellos",new
InstanciaPagoRecurrente(120000,DateTime.Now.AddMonths(-10),
null)));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[3],
TipoGastos[3], "Mantenimiento",new InstanciaPagoRecurrente(4200,
DateTime.Now.AddMonths(-6), null)));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[4],
TipoGastos[4], "Internet",new InstanciaPagoRecurrente(1800,
DateTime.Now.AddMonths(-8), null)));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[5],
TipoGastos[5], "Limpieza",new InstanciaPagoRecurrente(3500,
DateTime.Now.AddMonths(-5), null)));

    // ===== 9 pagos RECURRENTES con fecha de fin en el FUTURO =====
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[6],
TipoGastos[0], "Licencia software",new
InstanciaPagoRecurrente(2200, DateTime.Now.AddMonths(-4),
DateTime.Now.AddMonths(8))));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[7],
TipoGastos[1], "Seguro flota",new InstanciaPagoRecurrente(9500,
DateTime.Now.AddMonths(-7), DateTime.Now.AddMonths(5))));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[8],
TipoGastos[2], "Alquiler depósito",new
InstanciaPagoRecurrente(54000, DateTime.Now.AddMonths(-2),
DateTime.Now.AddMonths(10))));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[9],
TipoGastos[3], "Servicio guardia",new
InstanciaPagoRecurrente(4100, DateTime.Now.AddMonths(-3),
DateTime.Now.AddMonths(4))));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[10],
TipoGastos[4], "Telefonía móvil",new
InstanciaPagoRecurrente(2600, DateTime.Now.AddMonths(-1),
DateTime.Now.AddMonths(12))));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[11],
TipoGastos[5], "Mantenimiento HVAC",new
InstanciaPagoRecurrente(7800, DateTime.Now.AddMonths(-6),
DateTime.Now.AddMonths(2))));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[12],
TipoGastos[0], "Soporte técnico",new

```

```
InstanciaPagoRecurrente(3300, DateTime.Now.AddMonths(-5),
DateTime.Now.AddMonths(6)));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[13],
TipoGastos[1], "Licencias antivirus",new
InstanciaPagoRecurrente(1450, DateTime.Now.AddMonths(-8),
DateTime.Now.AddMonths(3))));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[14],
TipoGastos[2], "Estacionamiento",new
InstanciaPagoRecurrente(2100, DateTime.Now.AddMonths(-2),
DateTime.Now.AddMonths(7))));

    // ===== 10 pagos RECURRENTES con fecha de fin en el PASADO =====
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[0],
TipoGastos[3], "Campaña marketing",new
InstanciaPagoRecurrente(12500, DateTime.Now.AddMonths(-14),
DateTime.Now.AddMonths(-6))));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[1],
TipoGastos[4], "Contrato limpieza",new
InstanciaPagoRecurrente(3800, DateTime.Now.AddMonths(-12),
DateTime.Now.AddMonths(-3))));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[2],
TipoGastos[5], "Consultoría RRHH",new
InstanciaPagoRecurrente(9200, DateTime.Now.AddMonths(-9),
DateTime.Now.AddMonths(-1))));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[3],
TipoGastos[0], "Capacitación personal",new
InstanciaPagoRecurrente(4600, DateTime.Now.AddMonths(-10),
DateTime.Now.AddMonths(-2))));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[4],
TipoGastos[1], "Servicio backup",new
InstanciaPagoRecurrente(1700, DateTime.Now.AddMonths(-11),
DateTime.Now.AddMonths(-5))));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[5],
TipoGastos[2], "Alquiler sala reuniones",new
InstanciaPagoRecurrente(3000, DateTime.Now.AddMonths(-7),
DateTime.Now.AddMonths(-4))));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[6],
TipoGastos[3], "Publicidad trimestral",new
InstanciaPagoRecurrente(8000, DateTime.Now.AddMonths(-8),
DateTime.Now.AddMonths(-2))));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[7],
TipoGastos[4], "Soporte ERP",new InstanciaPagoRecurrente(5100,
DateTime.Now.AddMonths(-15), DateTime.Now.AddMonths(-9))));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[8],
TipoGastos[5], "Servicio jardinería",new
InstanciaPagoRecurrente(2400, DateTime.Now.AddMonths(-13),
DateTime.Now.AddMonths(-8))));
```

```

    AltaPago(new Pago(MetodosPago.Credito, Usuarios[9],
TipoGastos[0], "Monitoreo cámaras",new
InstanciaPagoRecurrente(3500, DateTime.Now.AddMonths(-6),
DateTime.Now.AddMonths(-1))));

    // ===== 17 pagos ÚNICOS (con número de recibo agregado) =====
    AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[10],
TipoGastos[1], "Compra insumos oficina",new
InstanciaPagoUnico(2700, DateTime.Now.AddDays(-25), 2001));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[11],
TipoGastos[2], "Flete puntual mercadería",new
InstanciaPagoUnico(15400, DateTime.Now.AddDays(-10), 2002));
    AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[12],
TipoGastos[3], "Reparación impresora",new InstanciaPagoUnico(4300,
DateTime.Now.AddDays(-40), 2003));
    AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[13],
TipoGastos[4], "Catering reunión clientes",new
InstanciaPagoUnico(9600, DateTime.Now.AddDays(-5), 2004));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[14],
TipoGastos[5], "Taxi traslado urgente",new InstanciaPagoUnico(850,
DateTime.Now.AddDays(-3), 2005));
    AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[0],
TipoGastos[0], "Reposición sillas",new InstanciaPagoUnico(11800,
DateTime.Now.AddDays(-60), 2006));
    AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[1],
TipoGastos[1], "Papelería y timbres",new InstanciaPagoUnico(2100,
DateTime.Now.AddDays(-15), 2007));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[2],
TipoGastos[2], "Carga combustible única",new
InstanciaPagoUnico(4200, DateTime.Now.AddDays(-7), 2008));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[3],
TipoGastos[3], "Compra luces LED",new InstanciaPagoUnico(7800,
DateTime.Now.AddDays(-20), 2009));
    AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[4],
TipoGastos[4], "Pago peaje excepcional",new
InstanciaPagoUnico(340, DateTime.Now.AddDays(-1), 2010));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[5],
TipoGastos[5], "Servicio plomería",new InstanciaPagoUnico(5200,
DateTime.Now.AddDays(-13), 2011));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[6],
TipoGastos[0], "Compra monitor 27",new InstanciaPagoUnico(18500,
DateTime.Now.AddDays(-33), 2012));
    AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[7],
TipoGastos[1], "Cafetera nueva",new InstanciaPagoUnico(4200,
DateTime.Now.AddDays(-18), 2013));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[8],
TipoGastos[2], "Reemplazo neumático",new InstanciaPagoUnico(9300,
DateTime.Now.AddDays(-22), 2014));

```

```

        AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[9],
TipoGastos[3], "Servicio desinfección",new
InstanciaPagoUnico(6100, DateTime.Now.AddDays(-27), 2015)));
        AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[10],
TipoGastos[4], "Compra matafuegos",new InstanciaPagoUnico(3900,
DateTime.Now.AddDays(-45), 2016)));
        AltaPago(new Pago(MetodosPago.Debito, Usuarios[1],
TipoGastos[5], "Reparación notebook",new InstanciaPagoUnico(7200,
DateTime.Now.AddDays(-12), 2017)));
    }

    public Usuario ObtenerUsuario(string email, string
contrasena)
    {
        if (string.IsNullOrEmpty(email))
        {
            throw new Exception("El email y la contraseña no
pueden estar vacíos.");
        }
        if (!email.Contains("@") || !email.Contains("."))
        {
            throw new Exception("El email ingresado no es
válido.");
        }
        if (email.Length < 5)
        {
            throw new Exception("El email debe tener al menos 5
caracteres.");
        }
        if (!email.Contains("laEmpresa"))
        {
            throw new Exception("El email debe pertenecer a la
empresa (debe contener 'laEmpresa').");
        }
        if (string.IsNullOrEmpty(contrasena))
        {
            throw new Exception("El email y la contraseña no
pueden estar vacíos.");
        }
        foreach (Usuario usuario in Usuarios)
        {
            if (usuario.Email == email && usuario.Contrasenia
== contrasena)
            {
                return usuario;
            }
        }
    }

```

```

    }
    return null;
}

public void BajaGasto(TipoGasto tipoGasto)
{
    if(tipoGasto == null)
    {
        throw new Exception("El tipo de gasto no puede ser
nulo.");
    }
    try
    {
        TipoGastoTienePago(tipoGasto);
        tipoGasto.Activo = false;
    }
    catch (Exception e) {
        throw;
    }
}

public double GetTotalPagosUsuario(Usuario usuario)
{
    if (usuario == null)
    {
        throw new Exception("El usuario no puede ser
nulo.");
    }
    if (!Usuarios.Contains(usuario))
    {
        throw new Exception("El usuario no existe en el
sistema.");
    }
    if (Pagos.Count == 0)
    {
        throw new Exception("No hay pagos registrados en el
sistema.");
    }
    double total = 0;
    IEnumerable<Pago> pagosUsuario =
GetPagosUsuario(usuario);
    foreach (Pago pago in pagosUsuario)
    {
        total += pago.GetPagoMes();
    }
    return total;
}

```



```

        public IEnumerable<Pago> GetPagosUsuario(Usuario usuario,
DateTime? fechaPago = null)
        {
            if (usuario == null)
            {
                throw new Exception("El usuario no puede ser
nulo.");
            }
            if (!Usuarios.Contains(usuario))
            {
                throw new Exception("El usuario no existe en el
sistema.");
            }
            if (Pagos.Count == 0)
            {
                throw new Exception("No hay pagos registrados en el
sistema.");
            }
            List<Pago> pagosUsuario = new List<Pago>();
            foreach (Pago pago in Pagos)
            {
                if (pago.UsuarioAsociado.Email == usuario.Email &&
pago.EsPagoActivo(fechaPago ?? DateTime.Now))
                {
                    pagosUsuario.Add(pago);
                }
            }

            return pagosUsuario.AsEnumerable();
        }

        public IEnumerable<Pago> GetPagosMiembrosEquipo(string
nombreEquipo, Usuario user, DateTime? fechaPago = null)
        {
            Equipo e = GetEquipoPorNombre(nombreEquipo);
            if (e == null)
            {
                throw new Exception("El equipo no existe.");
            }
            if (e.GetMiembros().Count() == 0)
            {
                throw new Exception("El equipo no tiene
miembros.");
            }
            if (Pagos.Count == 0)
            {

```

```

        throw new Exception("No hay pagos registrados en el
sistema.");
    }
    List<Pago> pagosEquipo = new List<Pago>();
    foreach (Usuario u in e.GetMiembros())
    {
        if (u.Email == user.Email)
        {
            continue;
        }
        foreach (Pago p in GetPagosUsuario(u, fechaPago))
        {
            pagosEquipo.Add(p);
        }
    }
    pagosEquipo.Sort();
    return pagosEquipo;
}

```

```

public void TipoGastoTienePago(TipoGasto t)
{
    if (t == null)
    {
        throw new Exception("El tipo de gasto no existe.");
    }
    foreach (Pago p in Pagos)
    {
        if (p.UsaTipoGasto(t))
        {
            throw new Exception("No se puede eliminar, el
Tipo de gasto tiene un pago asociado.");
        }
    }
}

```

```

public void AltaPagoDesdeDTO(DTOpago dto, Usuario usuario)
{
    try
    {
        if (dto == null)
        {
            throw new Exception("El DTO de pago no puede
ser nulo.");
        }
    }
}

```

```

        dto.Validar();
    }
    catch (Exception ex)
    {
        throw new Exception("Error al validar el DTO de
pago: " + ex.Message);
    }

    TipoGasto tipoGasto =
GetTipoGastoByName(dto.TipoGasto);
    if (tipoGasto == null)
    {
        throw new Exception("El tipo de gasto no existe.");
    }
    MetodosPago metodoPago;
    if (!Enum.TryParse(dto.MetodosPago, out metodoPago))
    {
        throw new Exception("El método de pago no es
válido.");
    }
    InstanciaPago instanciaPago;
    if (dto.TipoPago == "Unico")
    {
        instanciaPago = new
InstanciaPagoUnico(dto.MontoBase, dto.FechaPago.Value,
dto.ReciboPago);
    }
    else if (dto.TipoPago == "Recurrente")
    {
        instanciaPago = new
InstanciaPagoRecurrente(dto.MontoBase, dto.FechaPago.Value,
dto.FechaFin);
    }
    else
    {
        throw new Exception("El tipo de pago no es
válido.");
    }

    Pago nuevoPago = new Pago(metodoPago, usuario,
tipoGasto, dto.Descripcion, instanciaPago);
    try
    {
        AltaPago(nuevoPago);
    }
    catch (Exception ex)
    {

```

```

        throw new Exception("Error al dar de alta el pago:
" + ex.Message);
    }
}
}
}
}

```

3.1.2 Usuarios

3.1.2.1 Clase Usuario

```

using Clases.Pagos;
using Clases.Roles;
using ObligatorioP2;
using System.Diagnostics.Metrics;

namespace Clases.Usuarios
{
    public class Usuario : IValidar , IComparable<Usuario>
    {
        public int Id { get; set; }
        public static int UId { get; set; } = 0;
        public string Nombre { get; set; }
        public string Apellido { get; set; }
        public string Contrasenia { get; set; }
        public string Email { get; set; }

        public Rol? MiRol { get; set; }
        public Equipo Equipo { get; set; }
        public DateTime FechaAlta { get; set; }
        public Usuario()
        {
            Id = UId++;
        }
        public Usuario(string nombre, string apellido, string
contrasenia, Equipo equipo, Rol? rol)
        {
            Id = UId++;
            Nombre = nombre;
            Apellido = apellido;
            Contrasenia = contrasenia;
            Equipo = equipo;
        }
    }
}

```

```
        FechaAlta = DateTime.Now;
        if (rol != null)
        {
            MiRol = rol;
        }
    }
    public override string ToString()
    {
        return $"Usuario: {Nombre} {Apellido}, Email: {Email},
Equipo: {Equipo.Nombre}";
    }
    public void Validar()
    {
        ValidarNombre();
        ValidarApellido();
        ValidarContrasenia();
        ValidarEquipo();
        ValidarFechaAlta();
    }
    private void ValidarNombre()
    {
        if (Nombre.Length < 3)
        {
            throw new Exception("El nombre debe tener al menos
3 caracteres.");
        }
    }
    public void SetRol( Rol rol)
    {
        if (rol == null)
        {
            throw new Exception("El rol no puede ser nulo.");
        }
        MiRol = rol;
    }
    private void ValidarApellido()
    {
        if (Apellido.Length < 3)
        {
            throw new Exception("El apellido debe tener al
menos 3 caracteres.");
        }
    }
    private void ValidarContrasenia()
    {
        if (Contrasenia.Length < 8)
        {

```

```

        throw new Exception("La contraseña debe tener al
menos 8 caracteres.");
    }
}
private void ValidarEquipo()
{
    if (Equipo == null)
    {
        throw new Exception("El equipo no Existe.");
    }
}
private void ValidarFechaAlta()
{
    if (FechaAlta == DateTime.MinValue)
    {
        throw new Exception("La fecha de alta no puede
estar vacia.");
    }
}
public string FirstThreeLetters(string str)
{
    if (str.Length < 3)
    {
        return str;
    }
    return str.Substring(0, 3);
}

public string CreateEmail(int counter)
{
    string email;
    if (counter > 0)
    {
        email = FirstThreeLetters(Nombre.ToLower()) +
FirstThreeLetters(Apellido.ToLower()) + counter +
"@laEmpresa.com";
        SetEmail(email);
        return email;
    }
    email = FirstThreeLetters(Nombre.ToLower()) +
FirstThreeLetters(Apellido.ToLower()) + "@laEmpresa.com";
    SetEmail(email);
    return email;
}
public void SetEmail(string email)
{
    ValidarEmail(email);
}

```

```

        Email = email;
    }
    public void ValidarEmail(string email)
    {
        if (email.Length < 10)
        {
            throw new Exception("El email no es valido.");
        }
        if (!email.Contains("@") || !email.Contains("."))
        {
            throw new Exception("El email no es válido.");
        }
    }

    public string GetEmail()
    {
        return Email;
    }
    public override bool Equals(object? obj)
    {
        if (obj is Usuario)
        {
            Usuario usuario = (Usuario)obj;
            return Id == usuario.Id || Email == usuario.Email;
        }
        return false;
    }

    public IEnumerable<Usuario> GetMiembrosEquipo()
    {
        if (Equipo == null)
        {
            throw new Exception("El usuario no pertenece a ningún equipo.");
        }
        if (Equipo.GetMiembros().Count() == 0)
        {
            throw new Exception("El equipo no tiene miembros.");
        }

        if (MiRol.VerMiembrosEquipo() == false)
        {
            throw new Exception("El usuario no tiene permiso para ver los miembros del equipo.");
        }
        return Equipo.GetMiembrosAsc();
    }

```

```

    }

    public string StringMiembrosEquipo()
    {
        return $"Usuario: {Nombre} {Apellido}, Email: {Email}";
    }

    public string GetNombreEquipo()
    {
        return Equipo.Nombre;
    }

    public int CompareTo(Usuario other)
    {
        return this.Email.CompareTo(other.Email);
    }
}

```

3.1.2.2 Clase Equipo

```

using Clases.Pagos;
using ObligatorioP2;
using System.ComponentModel.DataAnnotations;

namespace Clases.Usuarios
{
    public class Equipo : IValidar
    {
        public int Id { get; set; }
        public static int UId { get; set; } = 0;
        public string Nombre { get; set; }
        private List<Usuario> Miembros { get; set; } = new
List<Usuario>();

        public Equipo()
        {
            Id = UId++;
        }

        public Equipo(string nombre)
        {
            Id = UId;
            Nombre = nombre;
        }
    }
}

```



```

        UID++;
    }

    public override string ToString()
    {
        return $"Id: {Id}, Nombre: {Nombre}";
    }

    public string GetNombre()
    {
        return Nombre;
    }

    public void Validar()
    {
        ValidarNombre();
    }

    private void ValidarNombre()
    {
        if(Nombre.Length < 3)
        {
            throw new Exception("El nombre del equipo debe
tener al menos 3 caracteres.");
        }
    }

    public IEnumerable<Usuario> GetMiembros()
    {
        if (Miembros.Count == 0)
        {
            throw new Exception("El equipo no tiene
miembros.");
        }

        return Miembros;
    }

    public IEnumerable<Usuario> GetMiembrosAsc()
    {
        if (Miembros.Count == 0)
        {
            throw new Exception("El equipo no tiene
miembros.");
        }
        if (Miembros == null)
        {

```

```

        throw new Exception("La lista de miembros es
nula.");
    }
    List<Usuario> miembrosAsc = Miembros;
    miembrosAsc.Sort();
    return miembrosAsc;
}

public void AgregarMiembro(Usuario usuario)
{
    if (usuario == null)
    {
        throw new Exception("El usuario no puede ser
nulo.");
    }
    if (Miembros.Contains(usuario))
    {
        throw new Exception("El usuario ya es miembro del
equipo.");
    }
    Miembros.Add(usuario);
}

public override bool Equals(object? obj)
{
    if (obj is Equipo)
    {
        Equipo equipo = (Equipo)obj;
        return Id == equipo.Id || Nombre == equipo.Nombre;
    }
    return false;
}
}
}

```

3.1.2.3 Clase DTOUser

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Clases.Pagos;
using Clases.Usuarios;

```

```

namespace Clases.Usuarios
{
    public class DTOUser
    {
        public string nombreCompleto { get; set; }
        public string email { get; set; }
        public string equipo { get; set; }
        public string rol { get; set; }
        public DateTime fechaAlta { get; set; }
        public double totalMes { get; set; }
        public IEnumerable<Usuario>? MiembrosEquipo { get; set; } =
null;
        public IEnumerable<Pago> PagosEquipo { get; set; } = null;

        public IEnumerable<Pago> MisPagos { get; set; } = null;
        public DTOUser()
        {
        }
        public DTOUser(Usuario u)
        {
            nombreCompleto = u.Nombre + " " + u.Apellido;
            email = u.Email;
            equipo = u.Equipo.Nombre;
            rol = u.MiRol.GetNombreRol();
            fechaAlta = u.FechaAlta;
            totalMes = 0;
        }
    }
}

```

3.1.3 Pagos

3.1.3.1 Clase DTOPagos

```
namespace Clases.Pagos
{
    public class DTopago
    {
        public string MetodosPago { get; set; }
        public string TipoGasto { get; set; }
        public string Descripcion { get; set; }
        public double MontoBase { get; set; }
        public DateTime? FechaPago { get; set; }
        public DateTime? FechaFin { get; set; }
        public int ReciboPago { get; set; }
        public string TipoPago { get; set; }
        public DTopago()
        {
        }
        public DTopago(string metodosPago, string tipoGasto, string
descripcion, double montoBase, DateTime? fechaPago, int
reciboPago, string tipoPago, DateTime? fechaFin )
        {
            MetodosPago = metodosPago;
            TipoGasto = tipoGasto;
            Descripcion = descripcion;
            MontoBase = montoBase;
            FechaPago = fechaPago;
            ReciboPago = reciboPago;
            FechaFin = fechaFin;
            TipoPago = tipoPago;
        }
        public void Validar()
        {
            ValidarTipoPago();
            ValidarMetodosPago();
            ValidarTipoGasto();
            ValidarDescripcion();
            ValidarMontoBase();
            if (TipoPago == "Unico")
            {
                ValidarPagoUnico();
            }
            else if (TipoPago == "Recurrente")
            {
                ValidarPagoRecurrente();
            }
        }
        public void ValidarPagoUnico()
        {
            if (FechaPago == null)
```

```
        {
            throw new Exception("La fecha de pago no puede
estar vacia.");
        }
        if (ReciboPago <= 0)
        {
            throw new Exception("El numero de recibo debe ser
mayor a 0.");
        }
    }
    public void ValidarPagoRecurrente()
    {
        if (FechaPago == null)
        {
            throw new Exception("La fecha de inicio no puede
estar vacia.");
        }
        if (FechaFin != null)
        {
            if (FechaFin <= FechaPago)
            {
                throw new Exception("La fecha de fin debe ser
mayor a la fecha de inicio.");
            }
        }
    }
    public void ValidarMetodosPago()
    {
        if (string.IsNullOrWhiteSpace(MetodosPago))
        {
            throw new Exception("El metodo de pago no puede
estar vacio.");
        }
    }
    public void ValidarTipoGasto()
    {
        if (string.IsNullOrWhiteSpace(TipoGasto))
        {
            throw new Exception("El tipo de gasto no puede
estar vacio.");
        }
    }
    public void ValidarDescripcion()
    {
        if (string.IsNullOrWhiteSpace(Descripcion))
        {
            throw new Exception("La descripcion no puede estar
vacía.");
        }
    }
}
```

```

    }
}
public void ValidarMontoBase()
{
    if (MontoBase <= 0)
    {
        throw new Exception("El monto base debe ser mayor a 0.");
    }
}
public void ValidarTipoPago()
{
    if (string.IsNullOrEmpty(TipoPago))
    {
        throw new Exception("El tipo de pago no puede estar vacio.");
    }
}
}
}
}

```

3.1.3.2 Clase Instancia Pago (Abstracta)

```

using ObligatorioP2;

namespace Clases.Pagos
{
    public abstract class InstanciaPago : IValidar
    {
        public int Id { get; set; }
        public static int UId { get; set; } = 0;
        public double MontoBase { get; set; }

        public InstanciaPago()
        {
            Id = UId++;
        }
        public InstanciaPago(double montoBase)
        {
            Id = UId;
            MontoBase = montoBase;
            UId++;
        }
    }
}

```

```

        public double GetMontoBase()
        {
            return MontoBase;
        }

        public abstract double CalcularMontoPago(MetodosPago
metodoPago);

        public abstract bool EsPagoActivo(DateTime fecha);

        public abstract string MiTipo();

        public virtual void Validar()
        {
            ValidarMontoBase();
        }

        private void ValidarMontoBase()
        {
            if (MontoBase <= 0)
            {
                throw new Exception("El monto base debe ser mayor a
0.");
            }
        }

        public abstract int GetCuotas();
    }
}

```

3.1.3.3 Clase Instancia de pago Recurrente

```

namespace Clases.Pagos
{
    public class InstanciaPagoRecurrente : InstanciaPago
    {
        private DateTime FechaInicio { get; set; }
        private DateTime? FechaFin { get; set; }

        public InstanciaPagoRecurrente(double montoBase, DateTime
fechaInicio, DateTime? fechaFin = null) : base(montoBase)
        {

```

```

        FechaInicio = fechaInicio;
        if (fechaFin != null)
        {
            FechaFin = fechaFin;
        }

    }

    public DateTime GetFechaInicio()
    {
        return FechaInicio;
    }

    public DateTime? GetFechaFin()
    {
        return FechaFin;
    }

    public override void Validar()
    {
        base.Validar();
        ValidarFechaInicio();
        ValidarFechaFin();
    }

    public void ValidarFechaInicio()
    {
        if (FechaInicio == DateTime.MinValue)
        {
            throw new Exception("El campo fecha de inicio no
puede estar vacio.");
        }
    }

    public void ValidarFechaFin()
    {
        DateTime fechaInicio = GetFechaInicio();
        if (FechaFin < fechaInicio)
        {
            throw new Exception("La fecha fin no puede ser
anterior a la fecha de inicio.");
        }
    }

    private double CalcularRecargo()
    {
        int cuotasRestantes = GetCuotas();
        double recargo;
        switch (cuotasRestantes)
        {
            case <= 5:
                recargo = 1.03;

```



```

        break;
    case >= 10:
        recargo = 1.10;
        break;
    case >= 6 and <= 9:
        recargo = 1.05;
        break;
    }
    return recargo;
}

public override double CalcularMontoPago(MetodosPago
metodoPago)
{
    double montoBase = MontoBase;
    double recargo = CalcularRecargo();

    return montoBase * recargo;
}
public override int GetCuotas()
{
    if (FechaFin == null)
    {
        return -1; // Indica que el pago es indefinido
    }
    DateTime fechaFin = FechaFin ?? DateTime.Now;
    int meses = (fechaFin.Year - FechaInicio.Year) * 12 +
fechaFin.Month - FechaInicio.Month;
    return meses + 1; // +1 para incluir el mes de inicio
}
public int CuotasRestantes()
{
    if (FechaFin == null)
    {
        return -1; // Indica que el pago es indefinido
    }
    DateTime fechaFin = FechaFin ?? DateTime.Now;
    DateTime mesActual = DateTime.Now;

    int meses = (fechaFin.Year - mesActual.Year) * 12 +
fechaFin.Month - mesActual.Month;
    return meses + 1;
}
public override bool EsPagoActivo(DateTime fecha)
{
    return fecha >= FechaInicio && (FechaFin == null ||
fecha >= FechaInicio && fecha <= FechaFin);
}

```

```

    }
    public override string MiTipo()
    {
        return "Recurrente";
    }
    public override string ToString()
    {
        int mesesRestantes = CuotasRestantes();
        if (mesesRestantes < 0)
        {
            return $"Recurrente";
        }
        return $"Cuotas restantes {mesesRestantes}";
    }
}
}

```

3.1.3.4 Clase Instancia de pago Unico

```

namespace Clases.Pagos
{
    public class InstanciaPagoUnico : InstanciaPago
    {
        private DateTime FechaPago { get; set; }
        private int ReciboPago { get; set; }
        public InstanciaPagoUnico(double montoBase, DateTime
fechaPago, int reciboPago) : base(montoBase)
        {
            FechaPago = fechaPago;
            ReciboPago = reciboPago;
        }
        public override void Validar()
        {
            base.Validar();
            ValidarFechaPago();
            ValidarReciboPago();
        }
        public DateTime GetFechaPago()
        {
            return FechaPago;
        }
        public int GetReciboPago()
        {
            return ReciboPago;
        }
        private void ValidarFechaPago()
        {

```

```

        if (FechaPago == DateTime.MinValue)
        {
            throw new Exception("El campo fecha de pago no
puede estar vacio.");
        }
    }

    private void ValidarReciboPago()
    {
        if (ReciboPago <= 0)
        {
            throw new Exception("El numero de recibo debe ser
mayor a 0.");
        }
    }

    public override double CalcularMontoPago(MetodosPago
metodoPago)
    {
        if(metodoPago == MetodosPago.Efectivo)
        {
            return MontoBase * 0.8;
        }
        else
        {
            return MontoBase * 0.9;
        }
    }

    public override bool EsPagoActivo(DateTime fecha)
    {
        return FechaPago.Month == fecha.Month && FechaPago.Year
== fecha.Year;
    }

    public override string MiTipo()
    {
        return "Unico";
    }

    public override int GetCuotas()
    {
        return 1;
    }

    public override string ToString()
    {
        return "Pago Unico";
    }
}
}

```

3.1.3.5 Metodos de pago Enum

```
namespace Clases.Pagos
{
    public enum MetodosPago
    {
        Credito = 1,
        Debito = 2,
        Efectivo = 3
    }
}
```

3.1.3.6 Clase Pago

```
using Clases.Usuarios;
using ObligatorioP2;

namespace Clases.Pagos
{
    public class Pago : IValidar, IComparable<Pago>
    {
        public int Id { get; set; }
        public static int UId { get; set; }
        public MetodosPago MetodosPago { get; set; }
        public Usuario UsuarioAsociado { get; set; }
        public TipoGasto TipoGasto { get; set; }
        public string Descripcion { get; set; }
        public InstanciaPago InstanciaPago { get; set; }
        public double MontoFinal { get; set; }

        public Pago()
        {
            Id = UId++;
        }

        public Pago(MetodosPago metodosPago, Usuario
usuarioAsociado, TipoGasto tipoGasto, string descripcion,
InstanciaPago instanciaPago)
        {
            Id = UId++;
            MetodosPago = metodosPago;
            UsuarioAsociado = usuarioAsociado;
            TipoGasto = tipoGasto;
            Descripcion = descripcion;
            InstanciaPago = instanciaPago;
        }
    }
}
```

```

        SetMontoFinal();
    }

    public void Validar()
    {
        ValidarDescripcion();
        ValidarMetodosPago();
        ValidarTipoGasto();
        ValidarInstanciaPago();
        ValidarMontoFinal();
        ValidarUsuarioAsociado();
    }

    private void ValidarUsuarioAsociado()
    {
        if (UsuarioAsociado == null)
        {
            throw new Exception("El usuario asociado no puede
ser nulo");
        }
    }

    private void ValidarMontoFinal()
    {
        if (MontoFinal <= 0)
        {
            throw new Exception("El monto final debe ser mayor
a 0");
        }
    }

    private void ValidarInstanciaPago()
    {
        if (InstanciaPago == null)
        {
            throw new Exception("La instancia de pago no puede
ser nula");
        }
        InstanciaPago.Validar();
    }

    private void ValidarTipoGasto()
    {
        if (TipoGasto == null)
        {
            throw new Exception("El tipo de gasto no puede ser
nulo");
        }
    }

    private double SetMontoFinal()

```

```

    {
        try
        {
            if (InstanciaPago == null)
            {
                throw new Exception("La instancia de pago no
puede ser nula");
            }
            double montoFinal =
InstanciaPago.CalcularMontoPago(MetodosPago);
            MontoFinal = montoFinal;
            return MontoFinal;
        }
        catch (Exception ex)
        {
            throw new Exception("Error al calcular el monto
final: " + ex.Message);
        }
    }
    public double GetPagoMes()
    {
        if(InstanciaPago.GetCuotas() == -1)
        {
            return MontoFinal;
        }
        return MontoFinal / InstanciaPago.GetCuotas();
    }
    private void ValidarMetodosPago()
    {
        if (MetodosPago != MetodosPago.Credito && MetodosPago !=
MetodosPago.Debito &&
            MetodosPago != MetodosPago.Efectivo)
        {
            throw new Exception("Metodos pago invalido");
        }
    }

    private void ValidarDescripcion()
    {
        if (string.IsNullOrEmpty(Descripcion))
        {
            throw new Exception("La descripcion de puede estar
vacía");
        }
    }

    public override string ToString()
    {

```

```

        string texto = $"El Pago {Id} de monto {MontoFinal:F2}
fue abonado con el método \"{MetodosPago}\", por el usuario:
{(UsuarioAsociado != null ? UsuarioAsociado.ToString() :
"N/A")}. ";

        return texto + $" Detalles:
{InstanciaPago.ToString()}";
    }

    public bool EsPagoActivo(DateTime fecha)
    {
        return InstanciaPago.EsPagoActivo(fecha);
    }

    public override bool Equals(object? obj)
    {
        if (obj is Pago)
        {
            Pago pago = (Pago)obj;
            return Id == pago.Id;
        }
        return false;
    }

    public int CompareTo(Pago other)
    {
        if (other == null) return -1;
        double pagoMes = GetPagoMes();
        return pagoMes.CompareTo(other.GetPagoMes()) * -1;
    }

    public bool UsaTipoGasto(TipoGasto tipoGasto)
    {
        return TipoGasto.Equals(tipoGasto);
    }
}

```

3.1.3.7 Clase Tipo de Gasto

```

using ObligatorioP2;

namespace Clases.Pagos
{
    public class TipoGasto : IValidar
    {
        public int Id { get; set; }
        public static int UID {get; set;} = 0;
    }
}

```

```
public string Nombre { get; set; }
public string Descripcion { get; set; }

public bool Activo { get; set; } = true;
public TipoGasto()
{
    Id = UId++;
    Activo = true;
}

public TipoGasto(string nombre, string descripcion)
{
    Id = UId++;
    Nombre = nombre;
    Descripcion = descripcion;
    Activo = true;
}

public void Validar()
{
    ValidarNombre();
    ValidarDescripcion();
}

private void ValidarNombre()
{
    if (string.IsNullOrEmpty(Nombre))
    {
        throw new Exception("El nombre no puede estar
vacío");
    }
}

private void ValidarDescripcion()
{
    if (string.IsNullOrEmpty(Descripcion))
    {
        throw new Exception("La descripcion no puede estar
vacía");
    }
}

public override string ToString()
{
    return Nombre ;
}
```



```

        public override bool Equals(object? obj)
        {
            if (obj is TipoGasto)
            {
                TipoGasto tipoGasto = (TipoGasto)obj;
                return Nombre.ToLower() ==
tipoGasto.Nombre.ToLower() || Id == tipoGasto.Id;
            }
            return false;
        }
    }
}

```

3.1.4 Roles

3.1.4.1 Clase Rol

```

namespace Clases.Roles
{
    public abstract class Rol
    {
        public Rol()
        {
        }

        public abstract bool CargarNuevoPago();
        public abstract bool VerPagosMesActual();
        public abstract bool AddTipoGasto();
        public abstract bool RemoveTipoGasto();
        public abstract bool ListadoPagosEquipo();
        public abstract bool VerMiembrosEquipo();
        public abstract string GetNombreRol();
    }
}

```

3.1.4.2 Clase Rol Gerente

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

```

```

namespace Clases.Roles
{
    public class RolGerente : Rol
    {
        public RolGerente(): base() { }

        public override bool AddTipoGasto()
        {
            return true;
        }

        public override bool CargarNuevoPago()
        {
            return true;
        }

        public override bool ListadoPagosEquipo()
        {
            return true;
        }

        public override string ToString()
        {
            return "Gerente";
        }

        public override bool RemoveTipoGasto()
        { return true; }

        public override bool VerPagosMesActual()
        {
            return true;
        }

        public override bool VerMiembrosEquipo()
        {
            return true;
        }

        public override string GetNombreRol()
        {
            return "Gerente";
        }
    }
}

```

3.1.4.3 Clase Rol Empleado

```
using System;
```

```
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Clases.Roles
{
    public class RolEmpleado : Rol
    {
        public RolEmpleado() : base() { }
        public override bool AddTipoGasto()
        {
            return false;
        }

        public override bool CargarNuevoPago()
        {
            return true;
        }
        public override string ToString()
        {
            return "Empleado";
        }

        public override bool ListadoPagosEquipo()
        {
            return false; }

        public override bool RemoveTipoGasto()
        {
            return false;
        }

        public override bool VerPagosMesActual()
        {
            return true; }

        public override bool VerMiembrosEquipo()
        {
            return false;
        }

        public override string GetNombreRol()
        {
            return "Empleado";
        }
    }
}
```

```
}  
}
```

3.1.1.5 IValidar (Interface)

```
namespace ObligatorioP2;  
  
public interface IValidar  
{  
    public void Validar();  
}
```

3.2 Codigo MVC

3.2.1 Controllers

3.2.1.1 Empleado Controller

```
using Clases.Pagos;
using Clases.Usuarios;
using Microsoft.AspNetCore.Mvc;
using ObligatorioP2;
using WebApp_Op2.Filters;

namespace WebApp_Op2.Controllers
{
    public class EmpleadoController : Controller
    {
        Sistema sistema = Sistema.GetSistema();

        [LogActionFilter]
        [RolActionFilter()]

        public IActionResult Index()
        {
            return View();
        }

        [LogActionFilter]
        [RolActionFilter()]
        public IActionResult Perfil()
        {
            string userLogged =
HttpContext.Session.GetString("usuario");
            Console.WriteLine("Entré al perfil de Empleado");
            Usuario usuario = null;
            try
            {
                usuario = sistema.GetUsuarioPorEmail(userLogged);
            }
            catch (Exception e)
            {
                return RedirectToAction("Login", "Usuario");
            }
            DTOUser dtoUser = null;
            try
            {
                dtoUser = new DTOUser(usuario);
                dtoUser.totalMes =
sistema.GetTotalPagosUsuario(usuario);
            }
        }
    }
}
```

```

        catch (Exception e)
        {
            ViewBag.Error = e.Message;
            ViewBag.Pagos = new List<Pago>();
        }
        return View(dtoUser);
    }
    [LogActionFilter]
    [RolActionFilter()]
    public IActionResult MisPagos()
    {
        string userLogged =
HttpContext.Session.GetString("usuario");
        Usuario usuario = null;
        try
        {
            usuario = sistema.GetUsuarioPorEmail(userLogged);
        }
        catch (Exception e)
        {
            return RedirectToAction("Login", "Usuario");
        }
        try
        {
            ViewBag.Pagos = sistema.GetPagosUsuario(usuario);
        }
        catch (Exception e)
        {
            ViewBag.Error = e.Message;
            ViewBag.Pagos = new List<Pago>();
        }
        return View(usuario);
    }

    [LogActionFilter]
    [RolActionFilter()]
    public IActionResult AltaPago()
    {
        ViewBag.TiposGasto = sistema.GetTipoGastos();
        return View();
    }

    [LogActionFilter]
    [RolActionFilter()]
    [HttpPost]
    public IActionResult AltaPago(DTOpago dto)

```

```

        {
            string userLogged =
HttpContext.Session.GetString("usuario");
            Usuario usuario = null;
            try
            {
                usuario = sistema.GetUsuarioPorEmail(userLogged);
            }
            catch (Exception e)
            {
                return RedirectToAction("Login", "Usuario");
            }
            try
            {
                dto.Validar();
            }
            catch (Exception e)
            {
                ViewBag.Error = e.Message;
                return View();
            }
            if (!usuario.MiRol.CargarNuevoPago())
            {
                return RedirectToAction("Index", "Home");
            }
            try
            {
                sistema.AltaPagoDesdeDTO(dto, usuario);
            }
            catch (Exception e)
            {
                ViewBag.Error = e.Message;
                return View();
            }
            return RedirectToAction("MisPagos");
        }
    }
}

```

3.2.1.2 Gerente Controller

```

namespace WebApp_Op2.Controllers
{
    public class GerenteController : Controller

```

```

{
    Sistema sistema = Sistema.GetSistema();

    [LogActionFilter]
    [RolActionFilter()]

    public IActionResult Index()
    {
        return View();
    }

    [LogActionFilter]
    [RolActionFilter()]
    public IActionResult Gastos()
    {
        ViewBag.Error = TempData["Error"];
        ViewBag.Succes = TempData["Succes"];

        IEnumerable<TipoGasto> listaGastos =
sistema.GetTipoGastos();
        return View(listaGastos);
    }

    [LogActionFilter]
    [RolActionFilter()]
    public IActionResult Perfil()
    {
        string userLogged =
HttpContext.Session.GetString("usuario");
        Console.WriteLine("Entré al perfil del Gerente");

        Usuario usuario = null;
        try
        {
            usuario = sistema.GetUsuarioPorEmail(userLogged);
        }
        catch (Exception e)
        {
            return RedirectToAction("Login", "Usuario");
        }
        IEnumerable<Usuario> miembrosEquipo = null;
        DTOUser dtoUser = null;
        try
        {
            miembrosEquipo = usuario.GetMiembrosEquipo();

```



```

        dtoUser = new DTOUser(usuario);
        dtoUser.totalMes =
sistema.GetTotalPagosUsuario(usuario);
        dtoUser.MiembrosEquipo = miembrosEquipo;

    }
    catch (Exception e)
    {
        ViewBag.Error = e.Message;
        dtoUser = new DTOUser(usuario);
        dtoUser.MiembrosEquipo = new List<Usuario>();
    }

    return View(dtoUser);
}

[LogActionFilter]
[RolActionFilter()]
public IActionResult AltaGasto()
{
    return View();
}

[LogActionFilter]
[RolActionFilter()]
[HttpPost]
public IActionResult AltaGasto(TipoGasto tipoGasto)
{
    try
    {
        sistema.AltaTipoGasto(tipoGasto);
        TempData["Succes"] = "Tipo de gasto creado con
exito";

        return RedirectToAction("Gastos", "Gerente");
    }
    catch (Exception e)
    {
        ViewBag.Msg = e.Message;
    }

    return View();
}

[LogActionFilter]
[RolActionFilter()]

```

```

public IActionResult EliminarGasto(int id)
{
    TipoGasto tipoGasto = sistema.GetTipoGasto(id);

    if (tipoGasto == null)
    {
        return View(tipoGasto);
    }
    try
    {
        sistema.TipoGastoTienePago(tipoGasto);
    }
    catch (Exception e)
    {
        TempData["Error"] = e.Message;
        return RedirectToAction("Gastos");
    }
    return View(tipoGasto);
}

[LogActionFilter]
[RolActionFilter()]
[HttpPost]
public IActionResult EliminarGasto(TipoGasto tg)
{
    string userLogged =
HttpContext.Session.GetString("usuario");
    Usuario usuario = null;
    try
    {
        usuario = sistema.GetUsuarioPorEmail(userLogged);
    }
    catch (Exception e)
    {
        return RedirectToAction("Login", "Usuario");
    }
    if(!usuario.MiRol.RemoveTipoGasto())
    {
        return RedirectToAction("Index", "Home");
    }
    try {
        TipoGasto tipoGasto = sistema.GetTipoGasto(tg.Id);
        sistema.BajaGasto(tipoGasto);
        TempData["Succes"] = "Tipo de gasto eliminado con
exito";
    }catch(Exception e) {
        ViewBag.Error = e.Message;
        return View();
    }
}

```

```

        return RedirectToAction("Gastos", "Gerente");
    }

    [LogActionFilter]
    [RolActionFilter()]
    public IActionResult AltaPago()
    {
        ViewBag.TiposGasto = sistema.GetTipoGastosActivos();
        return View();
    }

    [LogActionFilter]
    [RolActionFilter()]
    [HttpPost]
    public IActionResult AltaPago(DTOpago dto)
    {
        string userLogged =
HttpContext.Session.GetString("usuario");
        Usuario usuario = null;
        try
        {
            usuario = sistema.GetUsuarioPorEmail(userLogged);
        }
        catch (Exception e)
        {
            return RedirectToAction("Login", "Usuario");
        }
        try
        {
            dto.Validar();
        }
        catch (Exception e)
        {
            ViewBag.Error = e.Message;
            return View();
        }
        if(!usuario.MiRol.CargarNuevoPago())
        {
            return RedirectToAction("Index", "Home");
        }
        try {
            sistema.AltaPagoDesdeDTO(dto, usuario);
        }catch(Exception e) {
            ViewBag.Error = e.Message;
            return View();
        }
        return RedirectToAction("Pagos");
    }

```

```

    }
    [LogActionFilter]
    [RolActionFilter()]
    public IActionResult Pagos()
    {
        string userLogged =
HttpContext.Session.GetString("usuario");
        Usuario usuario = null;
        try
        {
            usuario = sistema.GetUsuarioPorEmail(userLogged);
        }
        catch (Exception e)
        {
            return RedirectToAction("Login", "Usuario");
        }

        IEnumerable<Usuario> miembrosEquipo = null;
        IEnumerable<Pago> pagosEquipo = new List<Pago>();
        DTOUser dtoUser = null;
        try
        {
            dtoUser = new DTOUser(usuario);
            pagosEquipo =
sistema.GetPagosMiembrosEquipo(usuario.GetNombreEquipo(),
usuario);

            dtoUser.MisPagos =
sistema.GetPagosUsuario(usuario);
            dtoUser.PagosEquipo = pagosEquipo;
            ViewBag.FechaFiltro = DateTime.Now;;
        }
        catch (Exception e)
        {
            dtoUser = new DTOUser(usuario);

            ViewBag.Error = e.Message;
            dtoUser.MisPagos = new List<Pago>();
            dtoUser.PagosEquipo = new List<Pago>();
            ViewBag.FechaFiltro = DateTime.Now;;

        }
        return View(dtoUser);
    }
    [LogActionFilter]
    [RolActionFilter()]
    [HttpPost]
    public IActionResult Pagos(DateTime fecha)

```

```

        {
            string userLogged =
HttpContext.Session.GetString("usuario");
            Usuario usuario = null;
            try
            {
                usuario = sistema.GetUsuarioPorEmail(userLogged);
            }
            catch (Exception e)
            {
                return RedirectToAction("Login", "Usuario");
            }

            IEnumerable<Usuario> miembrosEquipo = null;
            IEnumerable<Pago> pagosEquipo = new List<Pago>();
            DTOUser dtoUser = null;
            try
            {
                dtoUser = new DTOUser(usuario);
                pagosEquipo =
sistema.GetPagosMiembrosEquipo(usuario.GetNombreEquipo(), usuario,
fecha);
                dtoUser.MisPagos = sistema.GetPagosUsuario(usuario,
fecha);
                dtoUser.PagosEquipo = pagosEquipo;
                ViewBag.FechaFiltro = fecha;
            }
            catch (Exception e)
            {
                dtoUser = new DTOUser(usuario);

                ViewBag.Error = e.Message;
                dtoUser.MisPagos = new List<Pago>();
                dtoUser.PagosEquipo = new List<Pago>();
                ViewBag.FechaFiltro = fecha;
            }
            return View(dtoUser);
        }
    }
}

```

3.2.1.3 Home Controller

```

using Microsoft.AspNetCore.Mvc;
using ObligatorioP2;

```

```

using System.Diagnostics;
using WebApp_Op2.Filters;
using WebApp_Op2.Models;

namespace WebApp_Op2.Controllers;

public class HomeController : Controller
{
    Sistema sistema = Sistema.GetSistema();

    public IActionResult Index()
    {
        ViewBag.Usuario = HttpContext.Session.GetString("usuario");
        ViewBag.Rol = HttpContext.Session.GetString("rol");
        return View();
    }

    public IActionResult Privacy()
    {
        return View();
    }
}

```

3.2.1.4 Usuario Controller

```

using Clases.Usuarios;
using Microsoft.AspNetCore.Mvc;
using ObligatorioP2;

namespace WebApp_Op2.Controllers;

public class UsuarioController : Controller
{
    Sistema sistema = Sistema.GetSistema();

    public IActionResult Login()
    {
        if (HttpContext.Session.GetString("usuario") != null)
        {
            return RedirectToAction("Index", "Home");
        }
        return View();
    }

    [HttpPost]
    public IActionResult Login(string email, string contrasena)
    {
        Usuario usuario = null;
    }
}

```

```

        try
        {
            usuario = sistema.ObtenerUsuario(email, contrasena);
        }
        catch (Exception ex)
        {
            ViewBag.Error = ex.Message;
            return View();
        }

        if (usuario != null)
        {
            HttpContext.Session.SetString("usuario", usuario.Email);
            if (usuario.MiRol != null)
            {
                HttpContext.Session.SetString("rol",
usuario.MiRol.ToString());
            }
            return RedirectToAction("Index",
usuario.MiRol.ToString());
        }
        else
        {
            ViewBag.Error = "Nombre de usuario o contraseña
incorrectos.";
            return View();
        }
    }

    [HttpPost]
    public IActionResult Logout()
    {
        HttpContext.Session.Clear();
        return RedirectToAction("Index", "Home");
    }
}

```

3.3.2 Filtros

3.3.2.1 Log Action Filter

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Filters;

namespace WebApp_Op2.Filters
{
    public class LogActionFilter : ActionFilterAttribute
    {

```

```

        public override void
OnActionExecuting(ActionExecutingContext context)
        {
            // Obtener controller/action actuales
            string controller =
context.RouteData.Values["controller"]?.ToString() ?? "";
            string action =
context.RouteData.Values["action"]?.ToString() ?? "";

            if (string.Equals(controller, "Usuario",
StringComparison.OrdinalIgnoreCase)
                && string.Equals(action, "Login",
StringComparison.OrdinalIgnoreCase))
            {
                return;
            }

            // Si no hay usuario en sesión, redirigir al Login
(UsuarioController.Login)
            string userLogged =
context.HttpContext.Session.GetString("usuario");
            if (string.IsNullOrEmpty(userLogged))
            {
                context.Result = new
RedirectToActionResult("Login", "Usuario", null);
                return;
            }

            // Logging opcional
            Console.WriteLine($"[Before] Action:
{context.ActionDescriptor.DisplayName} - Usuario: {userLogged}");
        }

        public override void OnActionExecuted(ActionExecutedContext
context)
        {
            Console.WriteLine($"[After] Action:
{context.ActionDescriptor.DisplayName}");
        }
    }
}

```

3.3.2.2 Rol Action Filter

```

using Clases.Usuarios;
using Microsoft.AspNetCore.Mvc;

```



```

using Microsoft.AspNetCore.Mvc.Filters;
using ObligatorioP2;

namespace WebApp_Op2.Filters
{
    // Solución: Heredar de ActionFilterAttribute para poder
    // sobrescribir los métodos OnActionExecuting y OnActionExecuted
    public class RolActionFilter : ActionFilterAttribute
    {
        public override void
OnActionExecuting(ActionExecutingContext context)
        {
            Sistema s = Sistema.GetSistema();
            // Obtener controller/action actuales
            string controller =
context.RouteData.Values["controller"]?.ToString() ?? "";

            string userLogged =
context.HttpContext.Session.GetString("usuario");
            string roleLogged =
context.HttpContext.Session.GetString("rol");

            if (string.IsNullOrEmpty(userLogged) ||
string.IsNullOrEmpty(roleLogged))
            {
                context.Result = new
RedirectToActionResult("Login", "Usuario", null);
                return;
            }

            if (controller != roleLogged)
            {
                context.Result = new
RedirectToActionResult("Login", "Usuario", null);
                return;
            }
        }

        public override void OnActionExecuted(ActionExecutedContext
context)
        {
            Console.WriteLine($"[After] Action:
{context.ActionDescriptor.DisplayName}");
        }
    }
}

```

3.3.3 Vistas

3.3.3.1 Empleado

3.3.3.1.1 Alta de Pago

```
@using Clases.Pagos

@*
    For more information on enabling MVC for empty projects, visit
    https://go.microsoft.com/fwlink/?LinkID=397860
*@
@{
}

<section class="flex items-center justify-center bg-gray-800
py-8">
    <form asp-action="AltaPago" method="post" class="w-full max-w-lg
bg-white rounded-lg shadow-md p-8 space-y-6">
        @Html.AntiForgeryToken()

        <h2 class="text-2xl font-semibold text-gray-800
text-center">Alta Pago</h2>

        @if (!string.IsNullOrEmpty(ViewBag.Error as string))
        {
            <div class="bg-red-100 border border-red-300
text-red-700 px-4 py-2 rounded">
                @ViewBag.Error
            </div>
        }

        <div>
            <label for="MetodosPago" class="block text-sm
font-medium text-gray-700">Método de pago</label>
            <select id="MetodosPago" name="MetodosPago" required
class="mt-1 block w-full px-4 py-2 border
border-gray-300 rounded-md bg-white text-gray-800">
                <option value="">-- Seleccione --</option>
                <option value="Debito">Débito</option>
                <option value="Credito">Crédito</option>
                <option value="Efectivo">Efectivo</option>
            </select>
        </div>

        <div>
            <label for="TipoGasto" class="block text-sm font-medium
text-gray-700">Tipo de gasto</label>
            <select id="TipoGasto" name="TipoGasto" required
```

```

        class="mt-1 block w-full px-4 py-2 border
border-gray-300 rounded-md bg-white text-gray-800">
        <option value="">-- Seleccione --</option>
        @if (ViewBag.TiposGasto != null)
        {
            foreach (TipoGasto tg in
(IEnumerable<TipoGasto>)ViewBag.TiposGasto)
            {
                if(tg.Activo){
                    <option
value="@tg.Nombre">@tg.Nombre</option>
                }}
            }
        </select>
    </div>

    <div>
        <label for="Descripcion" class="block text-sm
font-medium text-gray-700">Descripción</label>
        <textarea id="Descripcion" name="Descripcion" rows="3"
required
            class="mt-1 block w-full px-4 py-2 border
border-gray-300 rounded-md bg-white text-gray-800"
            placeholder="Descripción del
pago"></textarea>
    </div>

    <div>
        <label for="MontoBase" class="block text-sm font-medium
text-gray-700">Monto</label>
        <input type="number" id="MontoBase" name="MontoBase"
step="0.01" min="0" required
            class="mt-1 block w-full px-4 py-2 border
border-gray-300 rounded-md bg-white text-gray-800"
            placeholder="0.00" />
    </div>

    <div>
        <label for="TipoPago" class="block text-sm font-medium
text-gray-700">Tipo de pago</label>
        <select id="TipoPago" name="TipoPago" required
            class="mt-1 block w-full px-4 py-2 border
border-gray-300 rounded-md bg-white text-gray-800">
            <option value="">-- Seleccione --</option>
            <option value="Unico">Único</option>
            <option value="Recurrente">Recurrente</option>
        </select>
    </div>

```

```

        <div>
            <label for="FechaPago" class="block text-sm font-medium
text-gray-700">Fecha (inicio / pago)</label>
            <input type="date" id="FechaPago" name="FechaPago"
                class="mt-1 block w-full px-4 py-2 border
border-gray-300 rounded-md bg-white text-gray-800" />
        </div>

        <div id="fechaFinContainer" class="hidden">
            <label for="FechaFin" class="block text-sm font-medium
text-gray-700">Fecha fin (recurrente)</label>
            <input type="date" id="FechaFin" name="FechaFin"
                class="mt-1 block w-full px-4 py-2 border
border-gray-300 rounded-md bg-white text-gray-800" />
        </div>

        <div id="reciboContainer" class="hidden">
            <label for="ReciboPago" class="block text-sm
font-medium text-gray-700">N° recibo (único)</label>
            <input type="number" id="ReciboPago" name="ReciboPago"
min="1"
                class="mt-1 block w-full px-4 py-2 border
border-gray-300 rounded-md bg-white text-gray-800"
                placeholder="Ej: 2001" />
        </div>

        <div class="pt-4">
            <button type="submit"
                class="w-full inline-flex justify-center py-2
px-4 border border-transparent rounded-md shadow-sm text-white
bg-indigo-600 hover:bg-indigo-700 font-medium">
                Crear pago
            </button>
        </div>
    </form>
</section>

<script>
    (function () {
        const tipoPago = document.getElementById('TipoPago');
        const fechaFinContainer =
document.getElementById('fechaFinContainer');
        const reciboContainer =
document.getElementById('reciboContainer');
        const fechaFinInput = document.getElementById('FechaFin');
        const reciboInput = document.getElementById('ReciboPago');
    })

```

```
const fechaPagoInput =
document.getElementById('FechaPago');

function updateVisibility() {
  const val = tipoPago.value;
  if (val === 'Unico') {
    reciboContainer.classList.remove('hidden');
    reciboInput.required = true;

    fechaFinContainer.classList.add('hidden');
    fechaFinInput.required = false;

    fechaPagoInput.required = true;
  } else if (val === 'Recurrente') {
    fechaFinContainer.classList.remove('hidden');
    fechaFinInput.required = false;

    reciboContainer.classList.add('hidden');
    reciboInput.required = false;

    fechaPagoInput.required = true;
  } else {
    fechaFinContainer.classList.add('hidden');
    reciboContainer.classList.add('hidden');
    fechaFinInput.required = false;
    reciboInput.required = false;
    fechaPagoInput.required = false;
  }
}

tipoPago.addEventListener('change', updateVisibility);
updateVisibility();
})();
</script>
```

3.2.3.1.2 Index

```
@{
}

<section class="flex items-center justify-center bg-gray-800 py-12">
    <div class="w-full max-w-4xl px-4">
        <div class="bg-white rounded-lg shadow-md p-6 mb-6">
            <div class="flex items-center justify-between">
                <div>
                    <h1 class="text-3xl font-semibold text-gray-800">Panel Empleado</h1>
                    <p class="text-sm text-gray-600 mt-1">Accesos rápidos para tus pagos</p>
                </div>
                <div class="text-right">
                    <a href="@Url.Action("Perfil", "Empleado")" class="inline-block px-4 py-2 bg-indigo-600 text-white rounded hover:bg-indigo-700">Mi perfil</a>
                </div>
            </div>
        </div>

        <div class="grid grid-cols-1 sm:grid-cols-2 gap-6">
            <a href="@Url.Action("MisPagos", "Empleado")" class="block bg-white rounded-lg shadow p-5 hover:shadow-lg transition">
                <div class="flex items-center justify-between">
                    <div>
                        <h3 class="text-lg font-medium text-gray-800">Mis pagos</h3>
                        <p class="text-sm text-gray-500 mt-1">Ver tus pagos activos</p>
                    </div>
                    <div class="text-indigo-600 text-3xl font-bold">→</div>
                </div>
            </a>

            <a href="@Url.Action("AltaPago", "Empleado")" class="block bg-white rounded-lg shadow p-5 hover:shadow-lg transition">
                <div class="flex items-center justify-between">
                    <div>
                        <h3 class="text-lg font-medium text-gray-800">Registrar pago</h3>
                        <p class="text-sm text-gray-500 mt-1">Crear un pago único o recurrente</p>
                    </div>
                </div>
            </a>
        </div>
    </div>
</section>
```

```

        <div class="text-indigo-600 text-3xl
font-bold">+</div>
    </div>
</a>
</div>

    <div class="mt-8 bg-white rounded-lg shadow p-6">
        <h4 class="text-lg font-semibold text-gray-800
mb-3">Información</h4>
        <p class="text-sm text-gray-600">Aquí puedes ver y
gestionar tus pagos. Si necesitas que el gerente revise algún
gasto, usa las opciones para notificar o solicitar revisión.</p>
    </div>
</div>
</section>

```

3.2.3.1.3 Mis Pagos

```

`@using Clases.Pagos
@model Clases.Usuarios.Usuario

<section class="flex items-center justify-center bg-gray-800
py-8">
    <div class="w-full max-w-6xl px-4">
        <div class="bg-white rounded-lg shadow-md p-6 mb-6">
            <div class="flex items-center justify-between">
                <div>
                    <h1 class="text-2xl font-semibold
text-gray-800">@Model.Nombre @Model.Apellido</h1>
                    <p class="text-sm text-gray-600">Equipo:
@Model.Equipo?.Nombre</p>
                    <p class="text-sm text-gray-600">Email:
@Model.Email</p>
                </div>
                <div class="text-right">
                    <a href="@Url.Action("Perfil", "Empleado")"
class="inline-block px-4 py-2 bg-indigo-600 text-white rounded
hover:bg-indigo-700">Volver al Perfil</a>
                </div>
            </div>
        </div>
    </div>

    @if (!string.IsNullOrEmpty(ViewBag.Error as string))
    {
        <div class="bg-red-100 border border-red-300
text-red-700 px-4 py-3 rounded mb-6">
            @ViewBag.Error
        </div>
    }

```

```

    }

    <div class="grid gap-6">
        <div>
            <div class="bg-white rounded-lg shadow p-4 mb-4">
                <h2 class="text-lg font-medium
text-gray-800">Mis pagos</h2>
            </div>

            @{
                // Materializar la colección para evitar
múltiples enumeraciones y nulls
                List<Pago> misPagos = (ViewBag.Pagos as
IEnumerable<Pago>)?.ToList() ?? new List<Pago>();
            }

            <!-- Contenedor con altura máxima 75vh y scroll
vertical -->
            <div class="space-y-4 max-h-[75vh] overflow-y-auto">
                @if (misPagos.Count > 0)
                {
                    foreach (Pago p in misPagos)
                    {
                        bool activo =
p.EsPagoActivo(DateTime.Now);
                        <div class="bg-white rounded-lg shadow
p-4">

                            <div class="flex items-start
justify-between">

                                <div>
                                    <div class="flex
items-center space-x-2">
                                        <span class="text-sm
font-semibold text-gray-700">@p.InstanciaPago.MiTipo()</span>
                                        <span class="text-xs
text-gray-500">· @p.TipoGasto?.Nombre</span>
                                    </div>
                                    <p class="text-gray-700
mt-2 font-medium">@p.Descripcion</p>
                                    <p class="text-sm
text-gray-600 mt-1">Método: <span
class="font-semibold">@p.MetodosPago</span></p>
                                </div>
                                <div class="text-right">
                                    <div class="text-xl
font-semibold text-gray-800">$ @p.MontoFinal.ToString("F2")</div>
                                    <div class="text-sm
text-gray-600"></div>

```



```


@if (activo)
    {
        <span
class="inline-block px-2 py-1 text-xs bg-green-100 text-green-800
rounded">Activo</span>
    }
    else
    {
        <span
class="inline-block px-2 py-1 text-xs bg-gray-100 text-gray-700
rounded">Inactivo</span>
    }
    </div>
</div>
</div>

<div class="mt-3 grid grid-cols-1
sm:grid-cols-3 gap-2 text-sm text-gray-600">
    <div>
        <div class="text-xs
text-gray-500">Pago mes</div>
        <div
class="font-medium">@p.GetPagoMes().ToString("F2")</div>
    </div>

    <div>
        <div class="text-xs
text-gray-500">Detalles</div>
        <div class="font-medium">
            @if (p.InstanciaPago is
Clases.Pagos.InstanciaPagoUnico unico)
            {
                <span>Fecha:
@unico.GetFechaPago().ToString("dd/MM/yyyy")</span>
                <br />
                <span>Recibo:
@unico.GetReciboPago()</span>
            }
            else if
(p.InstanciaPago is Clases.Pagos.InstanciaPagoRecurrente rec)
            {
                <span>Inicio:
@rec.GetFechaInicio().ToString("dd/MM/yyyy")</span>
                <br />
                <span>Fin:
@(rec.GetFechaFin() ?.ToString("dd/MM/yyyy") ??
"Indefinido")</span>
            }
        </div>
    </div>
</div>


```

```

        }
        else
        {
            <span> - </span>
        }
    </div>
</div>

    <div>
        <div class="text-xs
text-gray-500">Cuotas / Estado</div>
        <div class="font-medium">
            @if (p.InstanciaPago is
Clases.Pagos.InstanciaPagoRecurrente rec2)
            {
                int cuotas =
rec2.GetCuotas();
                <span>@(cuotas ==
-1 ? "Indefinido" : cuotas.ToString()) cuotas</span>
            }
            else
            {
                <span>Pago
único</span>
            }
        </div>
    </div>
</div>
</div>
    }
    else
    {
        <div class="bg-white rounded-lg shadow p-6
text-gray-600">No tienes pagos activos.</div>
    }
</div>
</div>
</div>
</section>

```

3.2.3.1.4 Perfil

```
@using Clases.Pagos
@using Clases.Usuarios
@{
    ViewData["Title"] = "Perfil";

    DTOUser dto = @Model;
    IEnumerable<Pago> pagos = ViewBag.Pagos as IEnumerable<Pago> ??
Enumerable.Empty<Pago>();
    string error = ViewBag.Error as string;
}
<div class="bg-gray-900 border border-gray-800 shadow-xl
rounded-2xl px-10 py-8">
<div class="w-full my-4">
    @if (!string.IsNullOrEmpty(error))
    {
        <div class="alert alert-warning">
            @error
        </div>
    }

    <div class="flex flex-col md:flex-row gap-8 md:items-stretch">
        <div class="flex-1">
            <h2 class="text-2xl font-semibold text-white mb-6
text-left">Perfil</h2>
            <div class="space-y-2 text-gray-200 text-sm">
                <p><span class="font-semibold">Nombre:</span>@dto.nombreCompleto</p>
                <p><span class="font-semibold">Rol:</span>@dto.rol</p>
                <p><span class="font-semibold">Email:</span> @dto.email</p>
                <p><span class="font-semibold">Fecha incorporación:</span>
@dto.fechaAlta</p>
                <p><span class="font-semibold">Equipo:</span> @dto.equipo</p>
            </div>
        </div>
        <div class="w-full md:w-1/3 flex">
            <div class="bg-gray-800 rounded-2xl px-8 py-5
text-center shadow-lg
w-full h-full flex flex-col
justify-center">
                <p class="text-gray-300 text-sm font-semibold
leading-tight">
                    Total gasto<br />del mes
                </p>
                <p class="text-red-400 text-3xl font-bold mt-2">
                    $@dto.totalMes
                </p>
            </div>
        </div>
    </div>
</div>
```

```

        </p>
    </div>
</div>

    <div class="my-6 border-t border-gray-700"></div>

</div>
</div>
</div>

```

3.3.3.2 Gerente

3.3.3.2.1 Alta de Gasto

```

@model dynamic

@{
    ViewBag.Title = "title";
    Layout = "_Layout";
}

<section class="min-h-screen flex items-center justify-center
bg-gray-800">
    <div class="w-full max-w-md flex flex-col items-center">
        <form asp-action="AltaGasto" method="post" class="w-full
max-w-md bg-white rounded-lg shadow-md p-8 space-y-6">

            <h2 class="text-2xl font-semibold text-gray-800
text-center">Alta de Gasto</h2>
            <div>
                <label for="nombre" class="block text-sm
font-medium text-gray-700">Nombre</label>
                <input
                    type="text"
                    id="nombre"
                    name="nombre"

                    class="mt-1 block w-full px-4 py-2 border
border-gray-300 rounded-md bg-white text-gray-800
placeholder-gray-400 focus:outline-none focus:ring-2
focus:ring-indigo-500 focus:border-indigo-500"
                    placeholder="Ingresar nombre"/>
            </div>

            <div>
                <label for="descripcion" class="block text-sm
font-medium text-gray-700">Descripcion</label>
                <input

```

```

        type="text"
        id="descripcion"
        name="descripcion"

        class="mt-1 block w-full px-4 py-2 border
border-gray-300 rounded-md bg-white text-gray-800
placeholder-gray-400 focus:outline-none focus:ring-2
focus:ring-indigo-500 focus:border-indigo-500"
        placeholder="Ingrese Descripcion"/>
    </div>

    <button
        type="submit"
        class="w-full inline-flex justify-center py-2 px-4
border border-transparent rounded-md shadow-sm text-white
bg-indigo-600 hover:bg-indigo-700 focus:outline-none focus:ring-2
focus:ring-offset-2 focus:ring-indigo-500 font-medium">
        Crear Gasto
    </button>

</form>
<p class="text-red-500 self-start mt-3"> @ViewBag.Msg</p>
</div>
</section>

```

3.3.3.2.2 Alta de Pago

```

@{
}
@using Clases.Pagos

@*
    For more information on enabling MVC for empty projects, visit
https://go.microsoft.com/fwlink/?LinkID=397860
*@
@{
}

<section class="flex items-center justify-center bg-gray-800
py-8">
    <form asp-action="AltaPago" method="post" class="w-full max-w-lg
bg-white rounded-lg shadow-md p-8 space-y-6">
        @Html.AntiForgeryToken()
    </form>
</section>

```

```

        <h2 class="text-2xl font-semibold text-gray-800
text-center">Alta Pago</h2>

        @if (!string.IsNullOrEmpty(ViewBag.Error as string))
        {
            <div class="bg-red-100 border border-red-300
text-red-700 px-4 py-2 rounded">
                @ViewBag.Error
            </div>
        }

        <div>
            <label for="MetodosPago" class="block text-sm
font-medium text-gray-700">Método de pago</label>
            <select id="MetodosPago" name="MetodosPago" required
                class="mt-1 block w-full px-4 py-2 border
border-gray-300 rounded-md bg-white text-gray-800">
                <option value="">-- Seleccione --</option>
                <option value="Debito">Débito</option>
                <option value="Credito">Crédito</option>
                <option value="Efectivo">Efectivo</option>
            </select>
        </div>

        <div>
            <label for="TipoGasto" class="block text-sm font-medium
text-gray-700">Tipo de gasto</label>
            <select id="TipoGasto" name="TipoGasto" required
                class="mt-1 block w-full px-4 py-2 border
border-gray-300 rounded-md bg-white text-gray-800">
                <option value="">-- Seleccione --</option>
                @if (ViewBag.TiposGasto != null)
                {
                    foreach (TipoGasto tg in
(IEnumerable<dynamic>)ViewBag.TiposGasto)
                    {
                        <option
value="@tg.Nombre">@tg.Nombre</option>
                    }
                }
            </select>
        </div>

        <div>
            <label for="Descripcion" class="block text-sm
font-medium text-gray-700">Descripción</label>
            <textarea id="Descripcion" name="Descripcion" rows="3"
required

```

```

        class="mt-1 block w-full px-4 py-2 border
border-gray-300 rounded-md bg-white text-gray-800"
        placeholder="Descripción del
pago"></textarea>
    </div>

    <div>
        <label for="MontoBase" class="block text-sm font-medium
text-gray-700">Monto</label>
        <input type="number" id="MontoBase" name="MontoBase"
step="0.01" min="0" required
        class="mt-1 block w-full px-4 py-2 border
border-gray-300 rounded-md bg-white text-gray-800"
        placeholder="0.00" />
    </div>

    <div>
        <label for="TipoPago" class="block text-sm font-medium
text-gray-700">Tipo de pago</label>
        <select id="TipoPago" name="TipoPago" required
        class="mt-1 block w-full px-4 py-2 border
border-gray-300 rounded-md bg-white text-gray-800">
            <option value="">-- Seleccione --</option>
            <option value="Unico">Único</option>
            <option value="Recurrente">Recurrente</option>
        </select>
    </div>

    <div>
        <label for="FechaPago" class="block text-sm font-medium
text-gray-700">Fecha (inicio / pago)</label>
        <input type="date" id="FechaPago" name="FechaPago"
        class="mt-1 block w-full px-4 py-2 border
border-gray-300 rounded-md bg-white text-gray-800" />
    </div>

    <div id="fechaFinContainer" class="hidden">
        <label for="FechaFin" class="block text-sm font-medium
text-gray-700">Fecha fin (recurrente)</label>
        <input type="date" id="FechaFin" name="FechaFin"
        class="mt-1 block w-full px-4 py-2 border
border-gray-300 rounded-md bg-white text-gray-800" />
    </div>

    <div id="reciboContainer" class="hidden">
        <label for="ReciboPago" class="block text-sm
font-medium text-gray-700">N° recibo (único)</label>

```

```

        <input type="number" id="ReciboPago" name="ReciboPago"
min="1"
        class="mt-1 block w-full px-4 py-2 border
border-gray-300 rounded-md bg-white text-gray-800"
        placeholder="Ej: 2001" />
    </div>

    <div class="pt-4">
        <button type="submit"
        class="w-full inline-flex justify-center py-2
px-4 border border-transparent rounded-md shadow-sm text-white
bg-indigo-600 hover:bg-indigo-700 font-medium">
            Crear pago
        </button>
    </div>
</form>
</section>

<script>
    (function () {
        const tipoPago = document.getElementById('TipoPago');
        const fechaFinContainer =
document.getElementById('fechaFinContainer');
        const reciboContainer =
document.getElementById('reciboContainer');
        const fechaFinInput = document.getElementById('FechaFin');
        const reciboInput = document.getElementById('ReciboPago');
        const fechaPagoInput =
document.getElementById('FechaPago');

        function updateVisibility() {
            const val = tipoPago.value;
            if (val === 'Unico') {
                reciboContainer.classList.remove('hidden');
                reciboInput.required = true;

                fechaFinContainer.classList.add('hidden');
                fechaFinInput.required = false;

                fechaPagoInput.required = true;
            } else if (val === 'Recurrente') {
                fechaFinContainer.classList.remove('hidden');
                fechaFinInput.required = false;

                reciboContainer.classList.add('hidden');
                reciboInput.required = false;

                fechaPagoInput.required = true;
            }
        }
    })();

```



```

        } else {
            fechaFinContainer.classList.add('hidden');
            reciboContainer.classList.add('hidden');
            fechaFinInput.required = false;
            reciboInput.required = false;
            fechaPagoInput.required = false;
        }
    }

    tipoPago.addEventListener('change', updateVisibility);
    updateVisibility();
  }) ();
</script>

```

3.3.3.2.3 Eliminar Gasto

```

@model Clases.Pagos.TipoGasto

@{
    ViewBag.Title = "title";
    Layout = "_Layout";
}
<h1>Eliminar Gasto</h1>

<form method="post">

    <label>Desea eliminar Gasto?</label>
    <br/>
    <br/>
    <button
        type="submit"
        class="w-full inline-flex justify-center py-2 px-4 border
border-transparent rounded-md shadow-sm text-white bg-indigo-600
hover:bg-indigo-700 focus:outline-none focus:ring-2
focus:ring-offset-2 focus:ring-indigo-500 font-medium">
        Eliminar
    </button>
</form>

```

3.3.3.2.4 Gastos

```
@using Clases.Pagos

@{

    Layout = "_Layout";
    int CantActivos = 0;

    foreach (TipoGasto tg in Model)
    {
        if (tg.Activo)
        {
            CantActivos++;
        }
    }

    string error = "";
    if (ViewBag.Error != null)
    {
        error = ViewBag.Error;
    }
    string succes = "";
    if (ViewBag.Succes != null)
    {
        succes = ViewBag.Succes;
    }
}

@if (Model != null)
{
    <div class="bg-gray-800 rounded-xl overflow-hidden shadow border
border-gray-700">
        <table class="w-full text-left">
            <thead class="bg-gray-700 text-gray-200 text-sm">
                <tr>
                    <th class="px-6 py-3 font-semibold">

                        Nombre

                    </th>
                    <th class="px-6 py-3 font-semibold">

                        Descripcion

                    </th>
                </tr>
            </thead>
            <tbody>
                @foreach (var item in Model)
                {
                    <tr>
                        <td class="px-6 py-3">@item.Nombre</td>
                        <td class="px-6 py-3">@item.Descripcion</td>
                    </tr>
                }
            </tbody>
        </table>
    </div>
}
```

```

        </th>
        <th class="px-6 py-3 font-semibold">Accion</th>
    </tr>
</thead>
<tbody>
    @if (CantActivos != 0)
    {
        @foreach (TipoGasto tg in Model)
        {
            @if (tg.Activo)
            {
                <tr class="hover:bg-white/10 transition">
                    <td class="p-4 border-b border-white/10">
                        <p class="text-sm font-medium">@tg.Nombre</p>
                    </td>
                    <td class="p-4 border-b border-white/10">
                        <p class="text-sm">@tg.Descripcion</p>
                    </td>
                    <td class="p-4 border-b border-white/10">
                        @Html.ActionLink("Eliminar", "EliminarGasto", new { id
= tg.Id })
                    </td>
                </tr>
            }
        }
    }
    else
    {
        <tr>
            <td colspan="3" class="p-4 text-center text-gray-300
border-b border-white/10">
                No hay gastos para mostrar.
            </td>
        </tr>
    }
</tbody>
</table>
</div>
<div class="p-4 text-center text-gray-300 border-white/10
h-16">
    <p id="pMensaje"
        class="text-md px-4 py-2 font-semibold ">
        @if (error.Length > 0 ) { @error }
        @if (succes.Length > 0) { @succes }
    </p>
</div>

```

```

}

<script>
  if(@error.Length >0 || @succes.Length >0){
    const el = document.getElementById("pMensaje");
    if(@error.Length >0){
      el.classList.add("text-white","bg-red-400","mb-6",
"rounded-xl")
    };
    if(@succes.Length >0){
      el.classList.add("text-white","bg-green-400","mb-6", "rounded-xl")
    };

    setTimeout(() => {
      if (el) {

        el.classList.add("hidden")

      };
    }, 3000);
  }
</script>

```

3.3.3.2.5 Index

```

@{
}
<section class="flex items-center justify-center bg-gray-800
py-12">
  <div class="w-full max-w-6xl px-4">
    <div class="bg-white rounded-lg shadow-md p-6 mb-6">
      <div class="flex items-center justify-between">
        <div>
          <h1 class="text-3xl font-semibold
text-gray-800">Panel Gerente</h1>
          <p class="text-sm text-gray-600 mt-1">Visión
general y accesos rápidos</p>
        </div>
        <div class="text-right">
          <a href="@Url.Action("Perfil", "Gerente")"
class="inline-block px-4 py-2 bg-indigo-600 text-white rounded
hover:bg-indigo-700">Mi perfil</a>
        </div>
      </div>
    </div>
  </div>

```

```

<div class="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4
gap-6">
    <a href="@Url.Action("Pagos", "Gerente")" class="block
bg-white rounded-lg shadow p-5 hover:shadow-lg transition">
        <div class="flex items-center justify-between">
            <div>
                <h3 class="text-lg font-medium
text-gray-800">Pagos</h3>
                <p class="text-sm text-gray-500 mt-1">Ver y
gestionar pagos propios y del equipo</p>
            </div>
            <div class="text-indigo-600 text-3xl
font-bold">→</div>
        </div>
    </a>

    <a href="@Url.Action("Gastos", "Gerente")" class="block
bg-white rounded-lg shadow p-5 hover:shadow-lg transition">
        <div class="flex items-center justify-between">
            <div>
                <h3 class="text-lg font-medium
text-gray-800">Tipos de gasto</h3>
                <p class="text-sm text-gray-500 mt-1">Crear
/ eliminar y revisar dependencias</p>
            </div>
            <div class="text-indigo-600 text-3xl
font-bold">→</div>
        </div>
    </a>

    <a href="@Url.Action("AltaPago", "Gerente")"
class="block bg-white rounded-lg shadow p-5 hover:shadow-lg
transition">
        <div class="flex items-center justify-between">
            <div>
                <h3 class="text-lg font-medium
text-gray-800">Nuevo pago</h3>
                <p class="text-sm text-gray-500
mt-1">Registrar pago único o recurrente</p>
            </div>
            <div class="text-indigo-600 text-3xl
font-bold">+</div>
        </div>
    </a>

    <a href="@Url.Action("Perfil", "Gerente")" class="block
bg-white rounded-lg shadow p-5 hover:shadow-lg transition">
        <div class="flex items-center justify-between">

```

```

        <div>
            <h3 class="text-lg font-medium
text-gray-800">Equipo</h3>
            <p class="text-sm text-gray-500 mt-1">Ver
miembros y roles</p>
        </div>
        <div class="text-indigo-600 text-3xl
font-bold">👥</div>
    </div>
</a>
</div>

    <div class="mt-8 grid grid-cols-1 lg:grid-cols-2 gap-6">
        <div class="bg-white rounded-lg shadow p-6">
            <h4 class="text-lg font-semibold text-gray-800
mb-3">Resumen rápido</h4>
            <p class="text-sm text-gray-600">Puedes ver pagos
activos, crear nuevos y administrar tipos de gasto desde las
opciones anteriores.</p>
        </div>

        <div class="bg-white rounded-lg shadow p-6">
            <h4 class="text-lg font-semibold text-gray-800
mb-3">Acciones frecuentes</h4>
            <ul class="text-sm text-gray-600 list-disc
list-inside space-y-2">
                <li>Agregar un tipo de gasto</li>
                <li>Registrar pago único o recurrente</li>
                <li>Revisar pagos del equipo</li>
            </ul>
        </div>
    </div>
</div>
</section>

```

3.3.3.2.6 Pagos

```

@using Clases.Pagos
@model Clases.Usuarios.DTOUser

<section class="flex items-center justify-center bg-gray-800
py-8">
    <div class="w-full max-w-6xl px-4">
        <div class="bg-white rounded-lg shadow-md p-6 mb-6">
            <div class="flex items-center justify-between">
                <div>
                    <h1 class="text-2xl font-semibold
text-gray-800">@Model.nombreCompleto</h1>

```

```

        <p class="text-sm text-gray-600">Equipo:
@Model.equipo</p>
        <p class="text-sm text-gray-600">Email:
@Model.email</p>
    </div>
    <div class="text-right">
        <a href="@Url.Action("Perfil", "Gerente")"
class="inline-block px-4 py-2 bg-indigo-600 text-white rounded
hover:bg-indigo-700">Volver al Perfil</a>
    </div>
</div>
</div>

@if (!string.IsNullOrEmpty(ViewBag.Error as string))
{
    <div class="bg-red-100 border border-red-300
text-red-700 px-4 py-3 rounded mb-6">
        @ViewBag.Error
    </div>
}

<!-- Contenedor con columnas; la columna principal tendrá
altura fija (75vh) y scroll -->
<div class="grid grid-cols-1 md:grid-cols-2 gap-6">
    <div class="">
        <div class="bg-white rounded-lg shadow p-4 mb-4">
            <h2 class="text-lg font-medium
text-gray-800">Mis pagos</h2>
        </div>

        @{
            // materializamos la colección para evitar
enumeraciones múltiples
            IEnumerable<Pago> misPagos =
(@Model.MisPagos)?.ToList() ?? new List<Pago>();
        }

        <!-- Este contenedor tiene altura 75vh y scrollbar
vertical si hay más elementos -->
        <div class="space-y-4 bg-transparent p-0
max-h-[75vh] overflow-y-auto">
            @if (misPagos.Count() > 0)
            {
                foreach (Pago p in misPagos)
                {
                    bool activo =
p.EsPagoActivo(DateTime.Now);

```

```

<div class="bg-white rounded-lg shadow
p-4">
    <div class="flex items-start
justify-between">
        <div>
            <div class="flex
items-center space-x-2">
                <span class="text-sm
font-semibold text-gray-700">@p.InstanciaPago.MiTipo()</span>
                <span class="text-xs
text-gray-500">· @p.TipoGasto?.Nombre</span>
            </div>
            <p class="text-gray-700
mt-2 font-medium">@p.Descripcion</p>
            <p class="text-sm
text-gray-600 mt-1">Método: <span
class="font-semibold">@p.MetodosPago</span></p>
        </div>
        <div class="text-right">
            <div class="text-xl
font-semibold text-gray-800">@p.MontoFinal.ToString("F2")</div>
            <div class="text-sm
text-gray-600">/ monto total</div>
            <div class="mt-2">
                @if (activo)
                {
                    <span
class="inline-block px-2 py-1 text-xs bg-green-100 text-green-800
rounded">Activo</span>
                }
                else
                {
                    <span
class="inline-block px-2 py-1 text-xs bg-gray-100 text-gray-700
rounded">Inactivo</span>
                }
            </div>
        </div>
    </div>

    <div class="mt-3 grid grid-cols-1
sm:grid-cols-3 gap-2 text-sm text-gray-600">
        <div>
            <div class="text-xs
text-gray-500">Pago mes</div>
            <div
class="font-medium">@p.GetPagoMes().ToString("F2")</div>
        </div>

```



```

<div>
    <div class="text-xs
text-gray-500">Detalles</div>
    <div class="font-medium">
        @if (p.InstanciaPago is
Clases.Pagos.InstanciaPagoUnico unico)
        {
            <span>Fecha:
@unico.GetFechaPago().ToString("dd/MM/yyyy")</span>
            <br />
            <span>Recibo:
@unico.GetReciboPago()</span>
        }
        else if
(p.InstanciaPago is Clases.Pagos.InstanciaPagoRecurrente rec)
        {
            <span>Inicio:
@rec.GetFechaInicio().ToString("dd/MM/yyyy")</span>
            <br />
            <span>Fin:
@(rec.GetFechaFin()?.ToString("dd/MM/yyyy") ??
"Indefinido")</span>
        }
        else
        {
            <span> - </span>
        }
    </div>
</div>

<div>
    <div class="text-xs
text-gray-500">Cuotas / Estado</div>
    <div class="font-medium">
        @if (p.InstanciaPago is
Clases.Pagos.InstanciaPagoRecurrente rec2)
        {
            int cuotas =
rec2.GetCuotas();
            <span>@(cuotas ==
-1 ? "Indefinido" : cuotas.ToString()) cuotas</span>
        }
        else
        {
            <span>Pago
único</span>
        }
    </div>
</div>

```

```

        </div>
    </div>
</div>
</div>
    }
}
else
{
    <div class="bg-white rounded-lg shadow p-6
text-gray-600">No tienes pagos activos.</div>
}
</div>
</div>

<div class="md:col-span-1">
    <div class="bg-white rounded-lg shadow p-4 flex
flex-col mb-4 gap-4">
        <div class="flex items-center justify-between">
            <h2 class="text-lg font-medium
text-gray-800">
                Pagos del equipo</h2>
            <form asp-action="Pagos" method="post"
class="flex items-center gap-2">
                <input
                    type="date"
                    name="fecha"
                    class="px-3 py-1.5 border
border-gray-300 rounded-md text-sm text-gray-700
focus:outline-none focus:ring-2
focus:ring-indigo-500 focus:border-indigo-500"/>

                <button
                    type="submit"
                    class="px-3 py-1.5 text-sm
bg-indigo-600 text-white rounded-md hover:bg-indigo-700
transition">
                    Filtrar
                </button>
            </form>
        </div>
    </div>

    <div class="md:col-span-1">
        <div class="bg-white rounded-lg shadow px-4
py-2 flex flex-col mb-4 gap-4">
            <p class="text-black">Mostrando datos de la
fecha :

```

```

@ViewBag.FechaFiltro.Date.ToString("dd/MM/yyyy")
        </p>
        </div>
    </div>

    @{
        IEnumerable<Pago> pagosEquipo =
        (@Model.PagosEquipo)?.ToList() ?? new List<Pago>();
    }

    <!-- altura limitada y scroll vertical -->
    <div class="bg-white rounded-lg shadow p-4
max-h-[75vh] overflow-y-auto">
        @if (!pagosEquipo.Any())
        {
            <div class="text-gray-600">No hay pagos de
los miembros del equipo.</div>
        }
        else
        {
            foreach (Pago p in pagosEquipo)
            {
                bool activo =
p.EsPagoActivo(DateTime.Now);
                <div class="border-b last:border-b-0
pb-3 mb-3">

                    <div class="flex items-start
justify-between">

                        <div>
                            <div class="text-sm
font-semibold text-gray-700">@p.UsuarioAsociado?.Nombre
@p.UsuarioAsociado?.Apellido</div>
                            <p class="text-sm
text-gray-600">@p.TipoGasto?.Nombre ·
@p.InstanciaPago.MiTipo()</p>
                        </div>
                        <div class="text-right">
                            <div class="text-lg
font-semibold text-gray-800">@p.MontoFinal.ToString("F2")</div>
                            <div class="text-sm
text-gray-600">@p.MetodosPago</div>
                            <div class="mt-2">
                                @if (activo)
                                {
                                    <span
class="inline-block px-2 py-1 text-xs bg-green-100 text-green-800
rounded">Activo</span>

```

```

    }
    else
    {
        <span
class="inline-block px-2 py-1 text-xs bg-gray-100 text-gray-700
rounded">Inactivo</span>
    }
</div>
</div>
</div>

<div class="mt-2 text-sm
text-gray-600">
    @if (p.InstanciaPago is
Clases.Pagos.InstanciaPagoUnico u)
    {
        <div>Fecha:
@u.GetFechaPago().ToString("dd/MM/yyyy") · Recibo:
@u.GetReciboPago()</div>
    }
    else if (p.InstanciaPago is
Clases.Pagos.InstanciaPagoRecurrente r)
    {
        <div>Inicio:
@r.GetFechaInicio().ToString("dd/MM/yyyy") · Fin:
@(r.GetFechaFin()?.ToString("dd/MM/yyyy") ?? "Indefinido")</div>
    }
</div>
</div>
}
</div>
</div>
</div>
</section>

```

3.3.3.2.7 Profil

```
@using Clases.Pagos
@using Clases.Usuarios
@{
    ViewData["Title"] = "Perfil";

    DTOUser dto = @Model;
    IEnumerable<Usuario> mEquipo = dto.MiembrosEquipo;
    string error = ViewBag.Error as string;
```

```

}
<div>
    @if (!string.IsNullOrEmpty(error))
    {
        <div class="mb-4 bg-yellow-500/20 text-yellow-300 border
border-yellow-500 px-4 py-3 rounded-lg">
            @error
        </div>
    }

    <div class="bg-gray-900 border border-gray-800 shadow-xl
rounded-2xl px-10 py-8">

        <div class="flex flex-col md:flex-row md:items-start
md:justify-between gap-8">

            <div>
                <h2 class="text-3xl font-semibold text-white
mb-6">Perfil</h2>

                <div class="space-y-2 text-gray-200 text-sm">
                    <p><span class="font-semibold">Nombre:</span>
@dto.nombreCompleto</p>
                    <p><span class="font-semibold">Rol:</span>
@dto.rol</p>
                    <p><span class="font-semibold">Email:</span>
@dto.email</p>
                    <p><span class="font-semibold">Fecha de
incorporación:</span> @dto.fechaAlta</p>
                    <p><span class="font-semibold">Equipo:</span>
@dto.equipo</p>
                </div>
            </div>

            <div class="md:self-start">
                <div class="bg-gray-800 rounded-2xl px-8 py-5 text-center
shadow-lg">
                    <p class="text-gray-300 text-sm font-semibold
leading-tight">
                        Total gasto<br />del mes
                    </p>
                    <p class="text-red-400 text-3xl font-bold mt-2">
                        $@dto.totalMes
                    </p>
                </div>
            </div>
        </div>
    </div>

```

```

</div>

<section class="mt-12">
    <h3 class="text-2xl font-semibold text-white mb-6">Miembros
del Equipo</h3>

    <div class="bg-gray-800 rounded-xl overflow-hidden shadow
border border-gray-700">
        <table class="w-full text-left">
            <thead class="bg-gray-700 text-gray-200 text-sm">
                <tr>
                    <th class="px-6 py-3
font-semibold">Usuario</th>
                    <th class="px-6 py-3 font-semibold">Rol</th>
                    <th class="px-6 py-3 font-semibold">Fecha
Incorporacion</th>
                    <th class="px-6 py-3 font-semibold">Email</th>
                    <th class="px-6 py-3
font-semibold">Equipo</th>

                </tr>
            </thead>
            <tbody class="text-gray-100 text-sm">
                @foreach (Usuario miembro in mEquipo)
                {
                    <tr class="border-t border-gray-700">
                        <td class="px-6 py-3">
                            @miembro.Nombre @miembro.Apellido
                        </td>
                        <td class="px-6 py-3">
                            @miembro.MiRol.ToString()
                        </td>
                        <td class="px-6 py-3">
                            @miembro.FechaAlta.ToString("dd/MM/yyyy")
                        </td>
                        <td class="px-6 py-3">
                            @miembro.Email
                        </td>
                        <td class="px-6 py-3">
                            @miembro.Equipo.Nombre
                        </td>
                    </tr>
                }
            </tbody>
        </table>
    </div>

```

```

        </table>
    </div>
</section>
</div>
</div>

```

3.3.3.3 Home

3.3.3.3.1 Index

```

@{
    ViewData["Title"] = "Home Page";
}
@{
    string user = ViewBag.Usuario;
    string rol = ViewBag.Rol;
}
<!--
<div class="text-center">
    <h1>TeamPay Manager @if (!string.IsNullOrEmpty(user)) {
        @user
    }
    @if (!string.IsNullOrEmpty(rol)) {
        @rol
    }
</h1>
</div>
-->

<div class="bg-gray-800 p-8 max-w-md mx-auto text-center">
    <h1 class="text-3xl font-bold text-white mb-4">
        TeamPay Manager
    </h1>

    <p class="text-gray-300 text-lg leading-relaxed">
        La plataforma que organiza y registra los pagos
        de cada equipo de forma simple y eficiente.
    </p>
</div>

```

3.3.3.4 Shared

3.3.3.4.1 Layout

```

@using Microsoft.AspNetCore.Http
<!DOCTYPE html>
<html lang="en">

```

```

<head>
    <meta charset="utf-8"/>
    <meta name="viewport" content="width=device-width,
initial-scale=1.0"/>
    <title>@ViewData["Title"] TeamPay Manager</title>
    <link rel="stylesheet" href="~/css/site.css"
asp-append-version="true"/>
</head>
<body class="bg-gray-800 text-white">
    <section class="grid grid-cols-1 md:grid-cols-3 min-h-screen
w-full">
        <!-- Sidebar (antes header) - ocupa 1/3 en md+ -->
        <aside class="col-span-1 bg-gray-800 border-r min-h-screen
p-6 flex flex-col">
            <!-- Logo / título en la parte superior -->
            <div class="flex-none mb-4 m-auto border-b-1 ">
                <a class="text-2xl font-semibold text-white"
asp-area="" asp-controller="Home" asp-action="Index">TeamPay
Manager</a>
            </div>

            @{
                string rol = Context.Session.GetString("rol");
            }
            @if (rol == "Gerente")
            {
                <nav class="flex-none md:flex-1 w-full h-2/3">
                    <ul id="nav-content" class="w-full ">
                        <li class="flex-1 flex items-center">
                            <a class="w-full px-3 py-2 text-base
font-medium text-white hover:bg-gray-700 block" asp-area=""
asp-controller="Gerente" asp-action="Index">Inicio</a>
                        </li>
                        <li class="flex-1 flex items-center">
                            <a class="w-full px-3 py-2 text-base
font-medium text-white hover:bg-gray-700 block" asp-area=""
asp-controller="Gerente" asp-action="Perfil">Perfil</a>
                        </li>
                        <li class="flex-1 flex items-center">
                            <a class="w-full px-3 py-2 text-base
font-medium text-white hover:bg-gray-700 block" asp-area=""
asp-controller="Gerente" asp-action="Pagos">Ver Pagos</a>
                        </li>
                        <li class="flex-1 flex items-center">
                            <a class="w-full px-3 py-2 text-base
font-medium text-white hover:bg-gray-700 block" asp-area=""
asp-controller="Gerente" asp-action="AltaPago">Cargar Nuevo
Pago</a>

```



```

        </li>
        <li class="flex-1 flex items-center">
            <a class="w-full px-3 py-2 text-base
font-medium text-white hover:bg-gray-700 block" asp-area=""
asp-controller="Gerente" asp-action="Gastos">Ver Tipos de
Gastos</a>

        </li>
        <li class="flex-1 flex items-center">
            <a class="w-full px-3 py-2 text-base
font-medium text-white hover:bg-gray-700 block" asp-area=""
asp-controller="Gerente" asp-action="AltaGasto">Cargar Nuevo
Gasto</a>

        </li>
        <!-- Añade más items aquí; cada <li> con
class="flex-1" ocupará espacio igual -->
    </ul>
</nav>

}
else if(rol == "Empleado")
{
    <nav class="flex-none md:flex-1 w-full h-2/3">
        <ul id="nav-content" class="w-full ">
            <li class="flex-1 flex items-center">
                <a class="w-full px-3 py-2 text-base
font-medium text-white hover:bg-gray-700 block" asp-area=""
asp-controller="Empleado" asp-action="Index">Inicio</a>
            </li>

            <li class="flex-1 flex items-center">
                <a class="w-full px-3 py-2 text-base
font-medium text-white hover:bg-gray-700 block" asp-area=""
asp-controller="Empleado" asp-action="Perfil">Perfil</a>
            </li>
            <li class="flex-1 flex items-center">
                <a class="w-full px-3 py-2 text-base
font-medium text-white hover:bg-gray-700 block" asp-area=""
asp-controller="Empleado" asp-action="MisPagos">Ver Pagos</a>
            </li>
            <li class="flex-1 flex items-center">
                <a class="w-full px-3 py-2 text-base
font-medium text-white hover:bg-gray-700 block" asp-area=""
asp-controller="Empleado" asp-action="AltaPago">Cargar Nuevo
Pago</a>
            </li>

            <!-- Añade más items aquí; cada <li> con
class="flex-1" ocupará espacio igual -->

```

```

        </ul>
    </nav>
}
else
{
    <!-- Menú central: ocupa 2/3 de la altura (h-2/3 ==
4/6) -->
    <nav class="flex-none md:flex-1 w-full h-2/3">
        <ul id="nav-content" class="w-full">
            <li class="flex-1 flex items-center">
                <a class="w-full px-3 py-2 text-base
font-medium text-white hover:bg-gray-700 block" asp-area=""
asp-controller="Home" asp-action="Index">Inicio </a>
            </li>
            <li class="flex-1 flex items-center">
                <a class="w-full px-3 py-2 text-base
font-medium text-white hover:bg-gray-700 block" asp-area=""
asp-controller="Home" asp-action="Privacy">Privacy</a>
            </li>
            <!-- Añade más items aquí; cada <li> con
class="flex-1" ocupará espacio igual -->
        </ul>
    </nav>
}

<!-- Logout / acción en la parte inferior -->
<div class="flex-none mt-4">
    <div class="flex items-center justify-center
md:justify-start">
        @{
            string usuario =
Context.Session.GetString("usuario");
        }
        @if (!string.IsNullOrEmpty(usuario))
        {
            <form asp-controller="Usuario"
asp-action="Logout" method="post" class="m-0 w-full">
                <button type="submit" class="w-full
inline-flex justify-center items-center px-3 py-2 border
border-transparent text-sm hover:text-gray-300 font-medium
rounded-md text-white bg-red-600 hover:bg-red-700
focus:outline-none focus:ring-2 focus:ring-offset-2
focus:ring-indigo-500 transition-all transition-duration-300">
                    Cerrar sesión
                </button>
            </form>
        }
    </div>
}
else

```

```

        {
            <a class="inline-flex w-full justify-center
items-center px-3 py-2 border border-transparent text-sm
font-medium rounded-md text-indigo-700 bg-indigo-100
hover:bg-indigo-200" asp-controller="Usuario"
asp-action="Login">Iniciar sesión</a>
        }
    </div>
</div>
</aside>

    <!-- Main: ocupa 2/3 en md+ y centra el contenido -->
    <main class="col-span-1 md:col-span-2 flex items-center
justify-center py-6">
        <div class="w-full max-w-4xl px-4">
            @RenderBody()
        </div>
    </main>
</section>

<script>
    // Toggle sencillo del menú móvil
    (function () {
        const btn = document.getElementById('nav-toggle');
        const content = document.getElementById('nav-content');
        const iconOpen = document.getElementById('icon-open');
        const iconClose = document.getElementById('icon-close');

        if (!btn || !content) return;

        btn.addEventListener('click', function () {
            const isHidden = content.classList.contains('hidden');
            if (isHidden) {
                content.classList.remove('hidden');
                iconOpen?.classList.add('hidden');
                iconClose?.classList.remove('hidden');
                btn.setAttribute('aria-expanded', 'true');
            } else {
                content.classList.add('hidden');
                iconOpen?.classList.remove('hidden');
                iconClose?.classList.add('hidden');
                btn.setAttribute('aria-expanded', 'false');
            }
        });
    })();
</script>

```

```
@await RenderSectionAsync("Scripts", required: false)
</body>
</html>
```

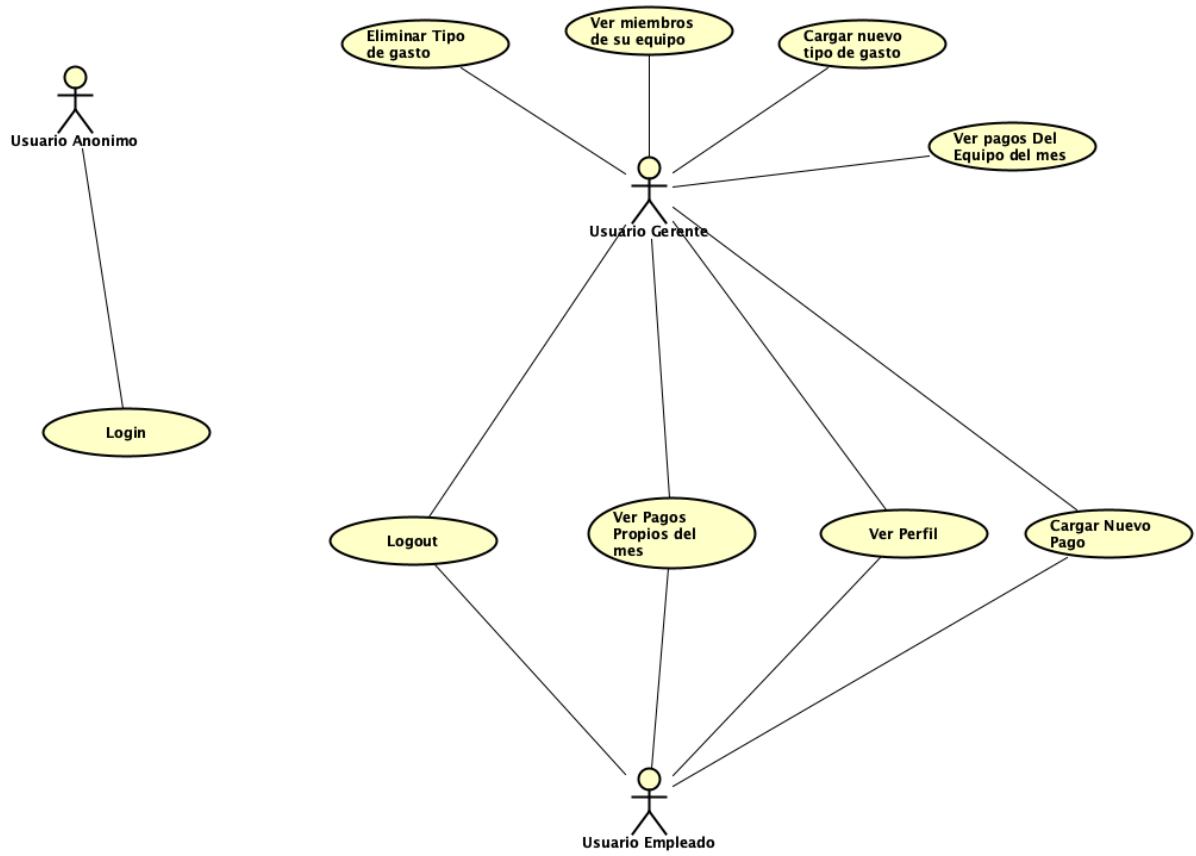
4.Prompts AI

Prompts utilizado para la precarga de datos >

Precarga Pagos: [Ver enlace](#)

Precarga Usuarios y Equipos: [Ver enlace](#)

5.Diagrama de casos de uso



6.Plantilla de casos de Prueba

6.1 Login

ID	Título	Criticidad	Precondición	Pasos	Resultado Esperado	Resultado Obtenido
LOG01	Login exitoso - Gerente	Alta	Usuario gerente existente con credenciales válidas	1. Abrir pantalla de login 2. Ingresar email válido de gerente 3. Ingresar contraseña válida (>=8 caracteres) 4. Ingresar	El sistema redirige al home del gerente y se cargan las funcionalidades correspondientes	Redirige a Vista gerente - PASA
LOG02	Login exitoso - Empleado	Alta	Usuario empleado existente con credenciales válidas	1. Abrir pantalla de login 2. Ingresar email válido de empleado 3. Ingresar contraseña válida (>=8 caracteres) 4. Ingresar	El sistema redirige al home del empleado y se cargan las funcionalidades correspondientes	Redirige a Vista empleado - PASA
LOG03	Login fallido - Contraseña incorrecta	Alta	Usuario existente	1. Abrir pantalla de login 2. Ingresar email válido 3. Ingresar contraseña incorrecta 4. Presionar Login	Se muestra mensaje de error: 'Credenciales inválidas'. No inicia sesión	Muestra Error: "Nombre de usuario o contraseña incorrectos" - PASA
LOG04	Login fallido - Usuario no registrado	Alta	Ninguna	1. Abrir pantalla de login 2. Ingresar email inexistente 3. Ingresar cualquier contraseña 4. Presionar Login	Se muestra error 'Usuario no Existe'. No inicia sesión	Muestra Error: "Nombre de usuario o contraseña incorrectos" - PASA
LOG05	Validación longitud mínima contraseña	Media	Usuario existente	1. Abrir pantalla de login 2. Ingresar email válido 3. Ingresar contraseña con menos de 8 caracteres 4. Ingresar	El sistema rechaza el login y muestra validación de longitud mínima	Muestra Error: "Nombre de usuario o contraseña incorrectos" - PASA
LOG06	Email vacío	Media	Ninguna	1. Abrir pantalla de login 2. Dejar email vacío 3. Ingresar contraseña válida 4. Ingresar	El sistema no permite enviar y muestra mensaje de validación	No permite ingresar - PASA
LOG07	Contraseña vacía	Media	Ninguna	1. Abrir pantalla de login 2. Ingresar email válido 3. Dejar contraseña vacía 4. Ingresar	El sistema no permite enviar y muestra mensaje de validación	No permite ingresar - PASA
LOG08	Redirección correcta según rol - Gerente	Alta	Usuario gerente logueado	1. Loguearse como gerente 2. Observar la redirección y funcionalidades	Se cargan solo las opciones del gerente	Se cargan las funcionalidades de Gerente - PASA
LOG09	Redirección correcta según rol - Empleado	Alta	Usuario empleado logueado	1. Loguearse como empleado 2. Observar la redirección y funcionalidades	Se cargan solo las opciones del empleado	Se cargan las funcionalidades de Empleado - PASA
LOG10	Formato de email inválido	Media	Ninguna	1. Abrir pantalla de login 2. Ingresar un email sin @ o dominio 3. Ingresar contraseña válida 4. Ingresar	Se muestra validación de formato incorrecto de email	No permite ingresar - PASA

6.2 Crear Nuevo Pago

ID	Título	Criticidad	Precondición	Pasos	Resultado Esperado	Resultado Obtenido
CP01	Pago Único - Débito/Crédito con descuento general (10%)	Alta	Usuario (empleado o gerente) logueado. Existen tipos de gasto configurados.	1. Ir a 'Crear Nuevo Pago'. 2. Seleccionar tipo de pago: Único. 3. Seleccionar método de pago: CRÉDITO o DÉBITO. 4. Seleccionar un tipo de gasto válido. 5. Ingresar descripción. 6. Ingresar monto = 1000. 7. Ingresar fecha de pago dentro del mes actual. 8. Ingresar número de recibo. 9. Crear Pago.	El pago se crea correctamente. - Monto total registrado = 1000. - Monto con beneficio = 900 (10% de descuento). - El sistema redirige al listado de pagos del usuario y el nuevo pago se visualiza en el mes actual.	Se muestra 900. PASA
CP02	Pago Único - Efectivo con descuento especial (20%)	Alta	Usuario (empleado o gerente) logueado. Existen tipos de gasto.	1. Ir a 'Crear Nuevo Pago'. 2. Seleccionar tipo de pago: Único. 3. Seleccionar método de pago: CRÉDITO o DÉBITO. 4. Seleccionar un tipo de gasto válido. 5. Ingresar descripción. 6. Ingresar monto = 1000. 7. Ingresar fecha de pago dentro del mes actual. 8. Ingresar número de recibo. 9. Crear Pago.	El pago se crea correctamente. - Monto total registrado = 1000. - Monto con beneficio = 800 (20% de descuento por EFECTIVO). - El sistema redirige al listado de pagos y el pago se visualiza en el mes actual.	Se muestra 800. PASA
CP03	Pago Recurrente con límite - Hasta 5 cuotas. 3% recargo.	Alta	Usuario logueado. El sistema calcula cantidad de meses entre fechas desde/hasta.	1. Ir a 'Crear Nuevo Pago'. 2. Seleccionar tipo de pago: Recurrente. 3. Ingresar fecha desde: 01/01/2025. 4. Ingresar fecha hasta: 01/05/2025 5. Seleccionar método de pago: CRÉDITO o DÉBITO. 6. Seleccionar tipo de gasto. 7. Ingresar descripción válida. 8. Ingresar monto mensual = 1000. 9. Crear Pago	El pago recurrente se crea correctamente. - Cantidad de meses del periodo = 5. - Monto total base = 5000. - Se aplica recargo del 3% sobre el total. - El monto con recargo = 5150. - El usuario es redirigido al listado de pagos.	Se muestra: 5 cuotas, 5150 moto total y 1030 cada cuota-PASA
CP04	Pago Recurrente con límite - Entre 6 y 9 cuotas. 5% recargo	Alta	Usuario logueado.	1. Ir a 'Crear Nuevo Pago'. 2. Seleccionar tipo: Recurrente. 3. Ingresar fecha desde: 01/01/2025. 4. Ingresar fecha hasta: 01/07/2025 5. Seleccionar método de pago: CRÉDITO o DÉBITO. 6. Seleccionar tipo de gasto. 7. Ingresar descripción válida. 8. Ingresar monto mensual = 1000. 9. Crear Pago	El pago recurrente se crea correctamente. - Cantidad de meses del periodo = 7. - Monto total base = 7000. - Se aplica recargo del 5% sobre el total. - El monto con recargo = 7350. - El usuario es redirigido al listado de pagos.	Se muestra: 7 cuotas, 7350 moto total y 1050 cada cuota-PASA
CP05	Pago Recurrente con límite - Más de 10 cuotas. 10% recargo	Alta	Usuario logueado.	1. Ir a 'Crear Nuevo Pago'. 2. Seleccionar tipo: Recurrente. 3. Ingresar fecha desde: 01/01/2025. 4. Ingresar fecha hasta: 01/12/2025 5. Seleccionar método de pago: CRÉDITO o DÉBITO. 6. Seleccionar tipo de gasto. 7. Ingresar descripción válida. 8. Ingresar monto mensual = 1000. 9. Crear Pago	El pago recurrente se crea correctamente. - Cantidad de meses = 12. - Monto total base = 12000. - Se aplica recargo del 10% por más de 10 cuotas. - Monto con recargo = 13200. - Redirección al listado de pagos tras la alta.	Se muestra: 12 cuotas, 13200 moto total y 1100 cada cuota-PASA

CP06	Pago Recurrente sin fecha de fin. Recargo 3%	Alta	Usuario logueado. El sistema permite dejar fecha hasta vacía para indicar que es sin límite.	1. Ir a 'Crear Nuevo Pago'. 2. Seleccionar tipo: Recurrente. 3. Ingresar fecha desde: dentro del mes actual. 4. Dejar fecha hasta vacía. 5. Seleccionar método de pago. 6. Seleccionar tipo de gasto. 7. Ingresar descripción válida. 8. Ingresar monto mensual = 1000. 9. Crear Pago	El pago recurrente sin límite se crea correctamente. - El sistema marca el pago como "Indefinido cuotas" - Se aplica recargo del 3% sobre el monto mensual al calcular el monto con recargo. - El pago queda activo para el mes actual. - Tras la alta, se redirige al listado de pagos.	Se muestra: fin: indefinido 1030 moto total y 1030 cada cuota-PASA
CP07	Pago Recurrente - Validación fecha fin anterior a fecha desde	Alta	Usuario logueado.	1. Ir a 'Crear Nuevo Pago'. 2. Seleccionar tipo: Recurrente. 3. Ingresar fecha desde: 10/05/2025. 4. Ingresar fecha hasta: 01/05/2025 (anterior). 5. Completar el resto de campos válidos. 6. Confirmar alta.	El sistema no permite crear el pago. - Muestra mensaje de error indicando que la fecha fin no puede ser anterior a la fecha desde.	No permite crear pago, Muestra error: "La fecha de fin debe ser mayor a la fecha de inicio." -PASA
CP08	Pago Recurrente - Validación de campos obligatorios (fecha desde, método, tipo gasto, monto)	Alta	Usuario logueado.	1. Ir a 'Crear Nuevo Pago'. 2. Seleccionar tipo: Recurrente. 3. Dejar fecha desde vacía. 4. No seleccionar método de pago. 5. No seleccionar tipo de gasto. 6. Ingresar monto vacío o 0. 7. Confirmar alta.	El sistema muestra mensajes de validación para todos los campos obligatorios: No se crea el pago.	No permite crear pago - PASA
CP09	Pago Único - Validar inclusión en listado del mes actual	Media	Usuario logueado.	1. Crear un pago único con fecha de pago dentro del mes actual y datos válidos. 2. Confirmar alta. 3. Verificar redirección automática al listado de pagos. 4. Verificar que el pago aparece en el listado del mes.	El pago único recién creado se visualiza en el listado de gastos del mes actual con: - Monto total correcto. - Monto con descuento de acuerdo al método de pago. - Usuario asociado correcto.	Se muestra en el listado luego de crearse el pago - PASA
CP10	Pago Recurrente - Validar inclusión como pago activo del mes actual (con fecha fin)	Alta	Usuario logueado. Fecha actual dentro del rango definido.	1. Crear un pago recurrente con fecha desde el 01/01/2025 y fecha hasta el 01/12/2025. 2. Confirmar alta. 3. Ir al listado de pagos 4. Verificar si el pago recurrente aparece en el listado.	Se muestra el monto de la cuota y el monto con recargo correspondiente y la cantidad de cuotas.	Se muestra Activo - PASA
CP11	Pago Recurrente - Validar inclusión como activo del mes actual (sin fecha fin)	Media	Usuario logueado. Fecha actual posterior a fecha desde.	1. Crear un pago recurrente sin fecha de fin, con fecha desde un mes anterior al actual. 2. Confirmar alta. 3. Ir al listado de pagos 4. Verificar que el pago aparece en el listado	El monto con recargo aplica el 3% definido para recurrentes sin límite.	Se muestra se muestran con recargo de 3 % - PASA
CP12	Flujo completo alta pago - Empleado redirigido a su listado	Alta	Usuario logueado con rol Empleado.	1. Ingresar como Empleado. 2. Navegar a 'Crear Nuevo Pago'. 3. Crear un pago válido (único o recurrente). 4. Confirmar alta.	Tras la creación del pago: - El sistema redirige automáticamente a la pantalla de 'Listado de pagos' del empleado.	Crea el pago correctamente - PASA
CP13	Flujo completo alta pago - Gerente redirigido a su listado	Alta	Usuario logueado con rol Gerente.	1. Ingresar como Gerente. 2. Navegar a 'Crear Nuevo Pago'. 3. Crear un pago válido. 4. Confirmar alta.	Tras la creación del pago: - El sistema redirige al listado de pagos accesible para el Gerente.	Crea el pago correctamente - PASA

6.3 Listado de Pagos

ID	Título	Criticidad	Precondición	Pasos	Resultado Esperado	Resultado Obtenido
LP01	Empleado - ver solo pagos del mes actual	Alta	Usuario empleado logueado con pagos en meses anteriores y en el mes actual	1. Ingresar como empleado 2. Navegar a 'Ver pagos'	Solo se muestran los pagos cuya fecha corresponde al mes y año actuales únicos y/o recurrentes. Pagos de otros meses no aparecen.	Solo muestra los pagos del empleado - PASA
LP02	Empleado - orden por monto total descendente usando cuota mensual	Alta	Empleado logueado con al menos un pago único y un pago recurrente activos este mes, con distintos montos	1. Ingresar como empleado 2. Navegar a 'Ver pagos'	El listado está ordenado por monto total descendente. Para pagos recurrentes se utiliza el valor de la cuota mensual para el orden.	Muestra en forma DESC - PASA
LP03	Empleado - no ve pagos de otros usuarios	Alta	Sistema con pagos de varios usuarios. Empleado logueado que tiene algunos pagos este mes.	1. Ingresar como empleado 2. Navegar a 'Ver pagos'	En el listado solo aparecen los pagos realizados por el usuario A. Pagos de otros usuarios no se muestran.	Ve solo sus pagos. PASA
LP04	Empleado - sin pagos en el mes actual	Media	Empleado logueado sin pagos en el mes y año actuales, pero con pagos en meses anteriores	1. Ingresar como empleado 2. Navegar a 'Ver pagos'	No se muestran registros	Ve lo del mes actual - PASA
LP05	Gerente - listado por defecto mes y año actuales	Alta	Gerente logueado con equipo que tiene pagos en el mes actual y en otros meses	1. Ingresar como gerente 2. Navegar a 'Ver pagos'	Se muestran los pagos del equipo y propios correspondientes al mes y año actuales, ordenados por monto total descendente.	Puede ver sus pagos y los del equipo - PASA
LP06	Gerente - filtrado por mes/año específicos	Alta	Gerente logueado con pagos de su equipo en distintos meses	1. Ingresar como gerente 2. Navegar a 'Ver pagos' 3. Ingresar un mes y año específicos 4. Aplicar filtro	El sistema muestra únicamente los pagos del equipo cuyo mes y año coinciden con el filtro ingresado.	Puede Filtrar segun mes y fecha, se ven los pagos segun la fecha de filtro - PASA
LP07	Gerente - Ver data de pagos equipo y personal	Alta	Gerente logueado con pagos en su equipo que tengan diferentes métodos de pago (crédito, débito, efectivo) y de beneficios/recargos	1. Ingresar como gerente 2. Navegar a 'Ver pagos' 3. Filtrar por mes actual	Para cada pago se muestra el monto total si es unico y si es recurrente el total y por cuota con beneficios/recargos aplicados según el método de pago.	Muestra info correcta - PASA
LP08	Gerente - sin pagos para mes filtrado	Media	Gerente logueado. No existen pagos del equipo o personales	1. Ingresar como gerente 2. Navegar a 'Ver pagos' 3. Filtrar por mes actual	El listado aparece vacío y se muestra un mensaje informativo de que no hay pagos para ese período.	Muestra texto que no tiene pagos. PASA
LP09	Acceso restringido a listado de equipo para no gerente	Alta	Usuario empleado logueado	1. Ingresar como empleado 2. Intentar acceder manualmente a la URL del listado de pagos de equipo (ej: /Gerente/Pagos)	El sistema impide el acceso y redirige a Login sin mostrar pagos de equipo.	No permite acceder - PASA

6.4 Nuevo tipo de Gasto

ID	Título	Criticidad	Precondición	Pasos	Resultado Esperado	Resultado Obtenido
TG01	Gerente crea nuevo tipo de gasto válido	Alta	Usuario gerente logueado. No existe un tipo de gasto con el nombre 'Cenas de empresa'.	1. Ingresar como gerente 2. Navegar a 'Cargar Nuevo gasto' 3. Ingresar nombre: 'Alquiler Oficina' 4. Ingresar descripción válida 5. Guardar	El sistema crea el tipo de gasto, redirige al listado de tipos de gasto y el nuevo registro aparece en la tabla.	Permite crear tipo de gasto - PASA
TG02	Empleado no puede acceder a creación de tipo de gasto	Alta	Usuario empleado logueado	1. Ingresar como empleado 2. Intentar acceder a la URL de gerente/AltaGasto	El sistema impide el acceso no muestra el formulario y redirige a Login.	No tiene y no puede acceder a la funcionalidad crear gasto - PASA
TG03	Validación de nombre obligatorio	Alta	Usuario gerente logueado	1. Ingresar como gerente 2. Navegar a 'Cargar Nuevo gasto' 3. Dejar el campo nombre vacío 4. Completar la descripción 5. Presionar Guardar	El sistema no crea el tipo de gasto, permanece en el formulario y muestra mensaje de 'Nombre obligatorio'.	Muestra error: "Error: El nombre no puede estar vacío" - PASA
TG04	Validación de descripción obligatoria	Alta	Usuario gerente logueado	1. Ingresar como gerente 2. Navegar a 'Cargar Nuevo gasto' 3. Completar nombre válido 4. Dejar descripción vacía 5. Guardar	El sistema no crea el registro, muestra mensaje de validación para descripción obligatoria y no redirige al listado.	Muestra error: "Error: La descripción no puede estar vacía" - PASA
TG05	No permitir tipo de gasto duplicado por nombre	Alta	Usuario gerente logueado. Ya existe tipo de gasto 'Alquiler'.	1. Ingresar como gerente 2. Navegar a 'Cargar Nuevo gasto' 3. Ingresar nombre: 'Alquiler' 4. Ingresar descripción válida 5. Guardar	El sistema rechaza la creación indicando que ya existe un tipo de gasto con ese nombre y permanece en el formulario.	Muestra error: "Error: El tipo de gasto ya existe." - PASA
TG06	Creación exitosa y disponibilidad en formulario Alta de Pago	Alta	Usuario gerente logueado.	1. Ingresar como gerente 2. Crear un nuevo tipo de gasto válido 'Suscripciones Software' 3. Confirmar y ser redirigido al listado 4. Navegar a 'Cargar Nuevo gasto' 5. Abrir el combo de tipos de gasto	El tipo de gasto 'Suscripciones Software' aparece disponible en el formulario de alta de gasto cuando se selecciona el tipo de gasto	Muestra el tipo de gasto creado previamente. - PASA

6.5 Eliminar Gasto

ID	Título	Criticidad	Precondición	Pasos	Resultado Esperado	Resultado Obtenido
ETG01	Gerente elimina tipo de gasto sin asociar a gasto	Alta	Usuario gerente logueado. Existe un tipo de gasto 'Alquiler' que no está asociado a ningún pago.	1. Ingresar como gerente 2. Ir a 'Ver tipo de gasto' 3. Ubicar 'Cafetería' 4. Hacer clic en 'Eliminar' 5. En la pantalla de confirmación, confirmar la eliminación	El sistema elimina el tipo de gasto 'Cafetería', redirige al listado y este ya no aparece en la tabla.	Elimina el gasto - PASA
ETG02	Gerente intenta eliminar tipo de gasto asociado a pago	Alta	Usuario gerente logueado. El tipo de gasto 'Sueldos' está asociado al menos a un pago.	1. Ingresar como gerente 2. Ir a 'Ver tipo de gasto' 3. Ubicar 'Sueldos' 4. Hacer clic en 'Eliminar' 5. Confirmar la eliminación	El sistema no elimina el tipo de gasto y muestra un mensaje de error indicando que no puede eliminarse porque está siendo utilizado por un pago.	No elimina el gasto. Muestra error: "No se puede eliminar, el Tipo de gasto tiene un pago asociado." - PASA
ETG03	Empleado no puede acceder a la opción de eliminar	Alta	Usuario empleado logueado.	1. Ingresar como empleado 2. Ir a 'Ver tipo de gasto' 3. Observar las acciones disponibles 4. Intentar acceder manualmente a la URL de eliminación	El empleado no ve el botón 'Eliminar' en la tabla y, si intenta acceder a la URL, el sistema lo redirige al login	No puede acceder - PASA
ETG04	Confirmación de eliminación muestra el tipo de gasto correcto	Media	Usuario gerente logueado. Existen varios tipos de gasto.	1. Ingresar como gerente 2. Ir al listado de tipos de gasto 3. Presionar 'Eliminar' sobre 'Viáticos' 4. Revisar la pantalla de confirmación	La pantalla de confirmación muestra claramente el nombre 'Viáticos' para evitar confusión antes de confirmar la eliminación.	Muestra pantalla de confirmación - PASA
ETG05	Validar que se elimine del form alta de pago	Alta	Usuario gerente logueado.	1. Ingresar como gerente 2. Ir al listado de tipos de gasto 3. Hacer clic en 'Eliminar' sobre 'Alquiler' 4. En la pantalla de confirmación, presionar 'Cancelar' e ir al form de Alta de pago	No se debe visualizar el gasto eliminado	No se muestra en Form de alta. - PASA
ETG06	Redirección al listado después de eliminar correctamente	Alta	Usuario gerente logueado.	1. Ingresar como gerente 2. Ir al listado de tipos de gasto 3. Eliminar un gasto	Tras la confirmación, la interfaz muestra nuevamente la tabla de tipos de gasto actualizada, sin el gasto eliminado	Redirección correcta. PASA

7.Link Azure

7.1 [click para ir a enlace Azure](#)